পর্ব ৩ · ডিজাইন · ক্লাস ১১ / ৩৭

ডেটা কীভাবে ভাবে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.১ (সিস্টেম ডিজাইন)
রিকোয়েস্ট ফ্লো · ১০ middleware · সেশন ও কুকি

৭ Django · ৩ নিজের · ৬টি স্তর

শুরু করুন প্রশ্ন দিয়ে

একটি পর্দা দেখা যাওয়ার আগে কতটা কাজ হয়?

হাত তুলুন: অনুমান করুন — ২টি ধাপ? ৫টি? ১০টি?

শিক্ষার্থীর ড্যাশবোর্ড খুলতে ১০টি middleware পার হয়, তারপর কুয়েরি, টেমপ্লেট, আর ৭৫টি HTML ফাইলের একটি।

এই যাত্রাটা না জানলে বাগ খুঁজতে গিয়ে ভুল জায়গায় খুঁজবেন। "লগইন হয়ে আবার লগআউট হয়ে যাচ্ছে" — সমস্যাটা view-এ নয়, **কুকিতে**। আজকের ক্লাসে এই যাত্রার প্রতিটি স্টেশন।

শিক্ষকের নোট: সংখ্যাটা জিজ্ঞেস করে হাত তুলিয়ে নিন — কেউ ১০ বলবে না। তারপর বলুন: "আপনি যেখানে বাগ খুঁজছিলেন, সেটা যাত্রার শেষ স্টেশন; সমস্যা ছিল প্রথম স্টেশনে।"

ধাপ ১

সিস্টেম ডিজাইন মানে ছয়টি প্রশ্নের উত্তর

প্রতিটি প্রশ্নের উত্তর একটা ডায়াগ্রামে আঁকা যায় — আর সেটাই ডিজাইন ডকুমেন্ট

প্রশ্ন	উত্তর দেয় কোন আর্টিফ্যাক্ট	IMS-এ কোথায়
কে ব্যবহার করবে?	ভূমিকা ম্যাট্রিক্স	পর্ব ২ ক্লাস ৯
ডেটা কোথায় থাকবে?	ERD ও স্কিমা	ক্লাস ১৪–১৫
ডেটা কীভাবে চলাচল করবে?	DFD Level 0 ও 1	ক্লাস ১৩
একটা অনুরোধ কীভাবে প্রক্রিয়া হবে?	**রিকোয়েস্ট ফ্লো**	**এই ক্লাস**
ব্যবহারকারী কী দেখবে?	wireframe ও লেআউট	ক্লাস ১২
ব্যর্থ হলে কী হবে?	ভুল-অবস্থা ও গার্ড	এই ক্লাস + ক্লাস ১৭

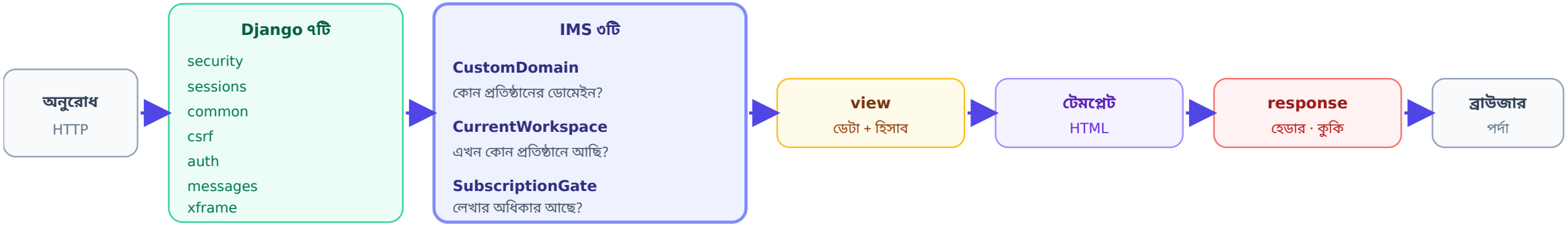
ছয়টির মধ্যে পাঁচটি ডায়াগ্রাম, একটি টেবিল। **কোনোটা লিখিত অনুচ্ছেদ নয়** — কারণ ডিজাইন দেখতে হয়, পড়তে হয় না।

শিক্ষকের নোট: ছাত্রদের বলুন এই ছয়টি প্রশ্নের উত্তর দিয়ে নিজের প্রজেক্টের ডিজাইন ডকুমেন্ট শুরু করতে — ৬টি পাতা, তার বেশি নয়।

ধাপ ২

Middleware কী — কেন এতগুলো

৭টি Django নিজের, ৩টি আমাদের — প্রতিটি একটা নির্দিষ্ট প্রশ্নের উত্তর দেয়



প্রতিটি middleware একই কাজ: অনুরোধ পেল →

হয় পাস করল, নয় থামাল/ঘুরাল। একটাও ডেটা বদলায় না, শুধু সিদ্ধান্ত নেয়।

শিক্ষকের নোট: বোর্ডে একটাই বাক্য লিখুন: "middleware = দরজার প্রহরী"। প্রতিটি প্রহরী একটাই প্রশ্ন করে, আর উত্তর না মিললে ভেতরে ঢুকতে দেয় না বা অন্য দরজায় পাঠায়।

ধাপ ৩

তিনটি প্রহরীর কাজ — এবং ক্রমের গুরুত্ব

`config/settings/base.py` — ক্রম বদলালে নিরাপত্তার ফাঁক

🌐

১. CustomDomain

হোস্টনাম দেখে বোঝে কোন প্রতিষ্ঠানের নিজের ডোমেইন। ভেরিফায়েড ডোমেইন হলে `ALLOWED_HOSTS`-এ যোগ করে।

না থাকলে: হোয়াইট-লেবেল অসম্ভব

🏢

২. CurrentWorkspace

সেশন থেকে বেছে নেয় "এখন কোন প্রতিষ্ঠানে আছি" — ভূমিকা ও ধরন অনুযায়ী সেরাটা। `request.workspace` বসায়।

না থাকলে: প্রতিটি view-তে ভুলে যাওয়ার ঝুঁকি

🔒

৩. SubscriptionGate

মেয়াদ শেষ হলে **শুধু লেখা** আটকায় — পড়া, লগআউট, প্ল্যাটফর্ম কনসোল খোলা থাকে।

না থাকলে: মেয়াদহীন প্রতিষ্ঠান অনির্দিষ্টকাল লেখে

ক্রম ভুলের পরিণাম: `SubscriptionGate` আগে বসালে সে জানে না কোন প্রতিষ্ঠান — কারণ `CurrentWorkspace` তখনো চলেনি। ফলে গেট চুপচাপ পাস করে দেয় → **নিরাপত্তার ফাঁক**, কোনো এরর নেই।

শিক্ষকের নোট: "চুপচাপ ফাঁক" ধারণাটা জোর দিন — সবচেয়ে বিপজ্জনক বাগ সেটাই, যেটা কোনো এরর দেয় না। ছাত্রদের বলুন প্রতি middleware-এ লিখতে: "আমি কার উপর নির্ভর করি?"

ধাপ ৪

সেশন ও কুকি — "লগইন হয়ে আবার লগআউট" বাগের আসল কারণ

প্রজেক্টের একমাত্র কমিট যেটা শুধু ডেমো-পরিবেশের জন্য

যা ঘটেছিল

ডেমো প্রিভিউ একটা **স্যান্ডবক্স iframe**-এ চলে, যেখানে কুকি রাখা যায় না।

ফল: লগইন সফল → সেশন কুকি হারিয়ে যায় → পরের পাতায় আবার লগইন পাতা।

ব্যবহারকারীর কাছে মনে হয় **পাসওয়ার্ড ভুল** — অথচ পাসওয়ার্ড ঠিক।

যা করা হলো (`e1ead8a`)

✓ কুকি-গার্ড ব্যানার — "এই প্রিভিউতে কুকি রাখা যায় না"

✓ ভদ্র CSRF এরর পাতা (৫০০ নয়)

✓ CSRF ব্যর্থ হলে কী করবেন তার নির্দেশ

এখান থেকে শেখার তিনটি কথা

১. সেশন = সার্ভারে ডেটা, কুকি = শুধু চাবি। কুকি না থাকলে সেশন খুঁজে পাওয়া যায় না।

২. পরিবেশ বদলালে **আচরণও** বদলায় — কোড একই থাকলেও।

৩. ব্যর্থতা **বোঝানো** যায়: ভদ্র বার্তা দেওয়া যায়, আর সেটাই ব্যবহারকারীর অভিজ্ঞতা।

সাধারণ নিয়ম: "কাজ করছে না" শুনলে প্রথম প্রশ্ন — **কোন পরিবেশে?** লোকাল, ডেমো, নাকি প্রোডাকশন।

শিক্ষকের নোট: নিজের অভিজ্ঞতা থেকে বলুন — এই বাগটা হয়েছিল বলে কুকি-গার্ড ব্যানার যোগ করতে হয়েছে। ছাত্রদের জিজ্ঞেস করুন: "তোমার প্রজেক্টে কোনো পরিবেশভেদে আচরণ বদলায়?"

ধাপ ৫

কোন প্রতিষ্ঠানে ঢুকব — সিদ্ধান্তের নিয়ম

`resolve_workspace()` — এক 함수, তিনটি সিদ্ধান্ত

```
def resolve_workspace(request):
    # ১. সেশনে সংরক্ষিত পছন্দ আছে?
    ws_id = request.session.get(
        "active_workspace_id")
    if ws_id and valid(ws_id):
        return membership        # পছন্দ মানুন

    # ২. নেই → সবচেয়ে উঁচু ভূমিকা
    membership = qs.order_by(
        ROLE_PRIORITY,       # ORG_ADMIN আগে
        KIND_PRIORITY,       # প্রতিষ্ঠান আগে
        "workspace__name")  # তারপর নাম
    request.session["active_workspace_id"]
        = membership.workspace_id
    return membership
```

তিনটি সিদ্ধান্ত, তিনটি কারণ

১. আগের পছন্দ থাকলে সেটাই — ব্যবহারকারী যেখানে ছিলেন সেখানেই ফিরবেন।

২. না থাকলে **উঁচু ভূমিকা** — যিনি স্কুল চালান, তিনি স্কুলেই ঢুকবেন, ব্যক্তিগত ঘরে নয়।

৩. তারপর **প্রতিষ্ঠান**, তারপর নাম — যাতে ফলাফল নির্দিষ্ট ও পুনরাবৃত্তিযোগ্য হয়।

> কোডের মন্তব্য হুবহু: *"the personal one is a side room, not the front door."*

শিক্ষকের নোট: তৃতীয় নিয়মটা ("তারপর নাম") বিশদ মনে হলেও জরুরি — এটা না থাকলে একই ইউজারের দুই প্রতিষ্ঠানের ডিফল্ট প্রতিবার বদলাতে পারে, আর ব্যবহারকারী ভাববেন "সিস্টেম এলোমেলো"।

ধাপ ৬

বিলিং গেট — মেয়াদ শেষ মানে "সব বন্ধ" নয়

পড়া খোলা, লেখা বন্ধ — আর বার্তাটা কে পাবে তা ভূমিকা ঠিক করে

পরিস্থিতি	গেট কী করে	কারণ
GET / পড়া	**পাস**	তথ্য দেখা বন্ধ করা মানে প্রতিষ্ঠানের ডেটা হারানো নয়, শুধু শাস্তি
POST ও লেখা	**বন্ধ**	মেয়াদ শেষে নতুন ডেটা জমা নেওয়া উচিত নয়
লগআউট · লগইন · স্ট্যাটিক · `/ops/`	**সবসময় পাস**	তালিকা: `ALLOW_PREFIXES` — নাহলে ব্যবহারকারী আটকে যান
অ্যাডমিন নিজে	**নবায়ন পাতায়**	শুধু সে-ই ঠিক করতে পারে → `?blocked=1`
বাকি সবাই	**ড্যাশবোর্ডে ফেরত**	বার্তা: "করুননি কিছু হারায়নি, আপনার অ্যাডমিনকে বলুন"

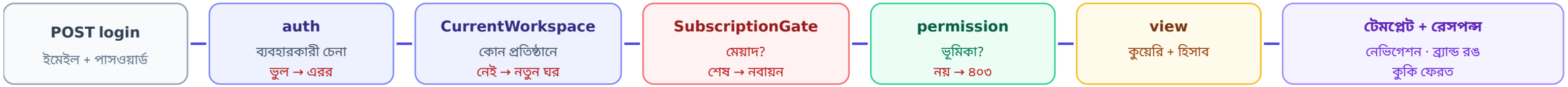
নীতি: মেয়াদ শেষ = **read-only**, delete নয়। ডেটা মুছে ফেলা মানে গ্রাহক ফিরে এলে কিছু দেওয়া নেই — আর সে ফিরবেই না।

শিক্ষকের নোট: প্রশ্ন করুন — "কেন সুপার-অ্যাডমিনের জন্য বাইপাস নেই?" কোডের নিজের উত্তর: যে প্ল্যাটফর্ম-অ্যাডমিন একই সাথে স্কুলের অ্যাডমিন, মেয়াদ শেষে সে-ও যেন লিখতে না পারে।

ধাপ ৭

পুরো যাত্রা — লগইন থেকে ড্যাশবোর্ড

প্রতিটি স্টেশনে কে সিদ্ধান্ত নেয়, আর ব্যর্থ হলে কী হয়



ড্যাশবোর্ড টেমপ্লেটে তিনটি জিনিস মেশে: নেভিগেশন (ভূমিকা অনুযায়ী) · ব্র্যান্ড (প্রতিষ্ঠানের রঙ) · ডেটা (view থেকে)।

রঙটা কোথা থেকে? base.html ফাইলে --ims-brand = প্রতিষ্ঠানের ব্র্যান্ড রঙ — তাই একই টেমপ্লেট, ভিন্ন প্রতিষ্ঠানে ভিন্ন রঙ।

ব্যর্থতার চারটি সম্ভাব্য জায়গা: auth · workspace · subscription · permission — চারটিই view-এর আগে।

তাই "পাতা খুলছে না" শুনলে প্রথমে view-এ নয়, এই চারটি প্রহরীর দিকে তাকান।

শিক্ষকের নোট: এই চারটি ব্যর্থতার জায়গা বোর্ডে আলাদা করে লিখুন। ছাত্ররা বাগ খুঁজতে সবসময় view-এ যায় — অথচ বাস্তবে বেশিরভাগ সমস্যা প্রথম চার স্টেশনে।

ধাপ ৮

রেসপন্সের ভেতরে কী থাকে

স্ট্যাটাস কোড, হেডার, কুকি — তিনটিই ব্যবহারকারীর আচরণ বদলায়

অংশ	IMS-এ উদাহরণ	ভুল হলে কী হয়
স্ট্যাটাস	৪০৪ অচেনা প্রতিষ্ঠান · ৪০৩ ভুল ভূমিকা · ৩০২ লগইনে পাঠানো	সব জায়গায় ৪০৩ দিলে প্রতিষ্ঠানের অস্তিত্ব ফাঁস
সিকিউরিটি হেডার	`X-Frame-Options: DENY` · `nosniff` · `Referrer-Policy`	কেউ আপনার সাইট অন্য সাইটের iframe-এ খুলে প্রতারণা করতে পারে
সেশন কুকি	প্রোডাকশনে `Secure` · `HttpOnly` · `SameSite=Lax`	স্ক্রিপ্ট দিয়ে কুকি পড়া যায় → অ্যাকাউন্ট দখল
ভ্যারি: Host	`patch_vary_headers(response, ["Host"])`	ক্যাশ ভিন্ন ডোমেইনের কনটেন্ট দেখাতে পারে
ব্র্যান্ড রঙ	`--ims-brand` = প্রতিষ্ঠানের রঙ	সব প্রতিষ্ঠানকে একই রকম দেখায় → হোয়াইট-লেবেল অর্থহীন

সবচেয়ে কম আলোচিত: **Vary: Host**। ক্যাশ সার্ভার হোস্ট আলাদা না করলে এক স্কুলের ডোমেইনে অন্য স্কুলের লোগো দেখাতে পারে — কোড ঠিক, ক্যাশ ভুল।

শিক্ষকের নোট: `deploy_check.sh`-এর আউটপুটটা এখানে মনে করিয়ে দিন — চারটি হেডার ওখানেই যাচাই হয়। বলুন: "হেডার লেখা মানে নয়, প্রমাণ করা লাগে।"

ধাপ ৯

স্কোপিং — টেন্যান্টের সীমানা কীভাবে রক্ষা হয়

প্রতিটি কুয়েরিতে প্রতিষ্ঠানের ফিল্টার — ভুলে যাওয়ার সুযোগ নেই

ঝুঁকিপূর্ণ পদ্ধতি

```
StudentProfile.objects.all()
# একটি view-তে filter ভুলে গেলেই
# অন্য প্রতিষ্ঠানের শিক্ষার্থী দেখা যাবে
```

ফুটো হয় না — **ফুটো হতে পারে**। এটাই ভয়ংকর।

নিরাপদ পদ্ধতি — `scoping.py`

```
students_in(request.workspace)
# ফিল্টার ভেতরেই — বাইরে ভুলে যাওয়া
# অসম্ভব
```

ভুল করা **অসম্ভব** বানানো — ভালো আর্কিটেকচারের লক্ষণ।

সোনার নিয়ম: ব্যবহারকারীর ইনপুট থেকে প্রতিষ্ঠান নেবেন না; `request.workspace` থেকে নিন — সেটা middleware যাচাই করে বসিয়েছে।

শিক্ষকের নোট: এই স্লাইডটা পর্ব ৪-এর ভিত্তি। ছাত্রদের বলুন আজই ঠিক করতে: তাদের প্রজেক্টে "এক প্রতিষ্ঠানের ডেটা" আনার একটাই ফাংশন থাকবে, আর সব view সেটাই ব্যবহার করবে।

ধাপ ১০

ডিজাইন ডকুমেন্টে যা আঁকবেন — এক পাতায়

চারটি বাক্স আর ছয়টি তির, তার বেশি নয়

থাকবে

- ✓ ব্রাউজার → প্রহরী → view → টেমপ্লেট
- ✓ প্রতিটি প্রহরীর **এক প্রশ্ন**
- ✓ ব্যর্থ হলে কোথায় যায় (৪০৩/৪০৪/৩০২)
- ✓ প্রতিষ্ঠান কোথা থেকে আসে
- ✓ কে ডেটাবেসে যায়, কে যায় না

থাকবে না

- ✗ প্রতিটি ফাংশনের নাম
- ✗ প্রতিটি টেবিলের ফিল্ড (ওটা ERD)
- ✗ CSS ক্লাস ও রঙের কোড
- ✗ লাইব্রেরির ভেতরের গঠন
- ✗ স্ক্রিনশট (ওটা wireframe-এর ক্লাস)

যাচাই: **এই ডায়াগ্রাম দেখে কেউ বলতে পারবে "পাতা খুলছে না" সমস্যাটা কোন বাক্সে?** না পারলে ডায়াগ্রামে ব্যর্থতার পথ আঁকা হয়নি।

শিক্ষকের নোট: ছাত্রদের বলুন ডায়াগ্রামে ব্যর্থতার তিরগুলো **লাল** রঙে আঁকতে — সফল পথ কালো, ব্যর্থ পথ লাল। এই এক অভ্যাসেই তাদের ডায়াগ্রাম অন্যদের চেয়ে বেশি কাজে লাগবে।

ক্লাসের কাজ

নিজের রিকোয়েস্ট ফ্লো আঁকুন — ২০ মিনিট

চারটি বাক্স, ছয়টি তির, দুই রঙ

১. চারটি বাক্স

ব্রাউজার · প্রহরী (একাধিক হতে পারে) · view · টেমপ্লেট

২. প্রহরীর প্রশ্ন

প্রতিটি বাক্সের নিচে এক লাইনে: "এই প্রহরী কী জিজ্ঞেস করে?"

৩. লাল তির

প্রতিটি প্রহরীর ব্যর্থতার পথ — কোথায় যায়, কী কোড।

৪. প্রতিষ্ঠান কোথা থেকে

URL? সেশন? টোকেন? — একটাই বেছে নিন, আর লিখে রাখুন।

৫. ক্যাশ ও কুকি

কোন ডেটা সেশনে থাকবে, কোনটা প্রতিবার ডেটাবেস থেকে আসবে।

যাচাই: আপনার ডায়াগ্রামে কি **একটি লাল তিরও** নেই? তাহলে আপনি ব্যর্থতার কথা ভাবেননি — আর বাস্তবে ঠিক ওখানেই বাগ হবে।

শিক্ষকের নোট: ২০ মিনিট শেষে যার ডায়াগ্রামে সবচেয়ে বেশি লাল তির, তারটাই বোর্ডে দেখান — বলুন "এটাই সবচেয়ে পরিণত ডিজাইন"।

আলোচনা

চারটি প্রশ্ন — রিকোয়েস্ট ফ্লো নিয়ে

উত্তরগুলো ক্লাস ১৭-এ কোড হিসেবে ফিরবে

১. আপনার সিস্টেমে **কোন প্রহরী কোন ক্রমে** বসবে? প্রতিটির এক প্রশ্ন লিখুন।

২. প্রতিষ্ঠান কোথা থেকে আসবে — URL, সেশন, নাকি দুটোই? দ্বন্দ্ব হলে কোনটা জিতবে?

৩. আপনার কোথায় **"ভুল করা অসম্ভব"** বানানো যায় — কোন কুয়েরিগুলো এক ফাংশনে ঢোকাবেন?

৪. আপনার সিস্টেম যদি কোনো **স্যান্ডবক্স iframe**-এ চলে, কী ভাঙবে? (প্রজেক্টে এটাই ঘটেছিল)

শিক্ষকের নোট: ৪ নম্বর প্রশ্নটা মজার কিন্তু গভীর — উত্তর দিতে গিয়ে ছাত্ররা বুঝবে কুকি-নির্ভর সিস্টেম কোথায় কোথায় ভাঙে। এই বোঝাটাই পরে সেশন বনাম টোকেনের সিদ্ধান্তে কাজে লাগবে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
এক পর্দার আগে ১০টি প্রহরী — ৭ Django + ৩ নিজের	`MIDDLEWARE`-এর হুবহু তালিকা, দশটি লাইন
প্রহরীর ক্রম নিরাপত্তার অংশ — ভুল ক্রমে চুপচাপ ফাঁক	CustomDomain → CurrentWorkspace → SubscriptionGate
পরিবেশ বদলালে আচরণ বদলায় — কোড একই, তবু	স্যান্ডবক্স iframe-এ কুকি থাকে না → `e1ead8a` কুকি-গার্ড
মেয়াদ শেষ = read-only, মুছে ফেলা নয়	`SAFE_METHODS` খোলা · `ALLOW_PREFIXES` তালিকা · বার্তা "আপনার কাজ হারায়নি"
ভুল করা অসম্ভব বানানো — ভুলে যাওয়ার উপর ভরসা নয়	`scoping.py` — ফিল্টার ফাংশনের ভেতরে, view-এ নয়

শিক্ষকের নোট: শেষ লাইনটা বারবার বলুন — ভালো সিস্টেম চেনা যায় এরকম প্রশ্নে: "এই ভুলটা করা কি সম্ভব?" উত্তর যদি "হ্যাঁ, যদি কেউ ভুলে যায়" হয়, ডিজাইনটা শেষ হয়নি।

১২

পরের ক্লাস — "স্ক্রিন আগে, কোড পরে"

wireframe ও UI: লেআউট · ভূমিকা অনুযায়ী পর্দা · খালি অবস্থা · ব্র্যান্ডিং

পর্ব ৩ · ডিজাইন · ক্লাস ১২ / ৩৭

স্ক্রিন আগে, কোড পরে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.১ (সিস্টেম ডিজাইন)

wireframe · ৪টি খোলস · ১৫টি টোকেন · খালি অবস্থা · white-label

৭৫ টেমপ্লেট · ৪ খোলস · ৬ partial

শুরু করুন প্রশ্ন দিয়ে

৭৫টি পর্দা লিখতে কতটা HTML লাগে?

হাত তুলুন: ৭৫টি ফাইল মানে ৭৫ বার sidebar লিখতে হয়েছে?

একবার sidebar লিখলে ৪৪টি পর্দায় দেখা যায়।
বাকিটা খোলস (shell) — পর্দা নয়, কাঠামো।

এই ক্লাসের বিষয় UI সাজানো, রং বাছাই নয়। কোন জিনিস **একবার** লিখে সব জায়গায় ব্যবহার করবেন, আর কোনটা পর্দাভেদে বদলাবে — এই সিদ্ধান্তটাই ডিজাইন।

শিক্ষকের নোট: ৭৫ সংখ্যাটা বোর্ডে লিখুন, নিচে ৪। বলুন: "পার্থক্যটা ৭১ নয় — পার্থক্যটা হলো ৪টি কাঠামো, যার ভেতরে ৭৫টি ঘর।"

ধাপ ১

চারটি খোলস — পুরো সিস্টেম এতেই ঢোকে

সবগুলো `base.html` থেকে আসে: শুধু `<head>` আর `<body>` ব্লক

base.html

head + body ব্লক · টোকেন

লগইন ৬ + এরর পাতা ৩

৯ পাতা

app_base.html

sidebar + topbar + content

আসল অ্যাপের সব পর্দা

৪৪ পাতা

platform/base.html

৬টি ট্যাব · নিজের রং

প্ল্যাটফর্ম অ্যাডমিনের ঘর

৮ পাতা

public/base_public.html

লগইনের আগের জগৎ

দাম · ট্রায়াল সাইনআপ

২ পাতা

৬টি partial টুকরো: sidebar · topbar · messages · billing_banner · field · cookie_guard — এগুলো খোলসের ভেতরে বসে।

> হোয়াইট-লেবেল: একই `app_base.html`, কিন্তু ব্র্যান্ড রং ভেরিয়েবল থেকে — তাই ভিন্ন প্রতিষ্ঠানে ভিন্ন রঙ, কোড এক।

শিক্ষকের নোট: চারটি বক্স বোর্ডে আঁকুন, তারপর বলুন: "নতুন পর্দা বানানোর সময় প্রশ্ন একটাই — এটা কোন খোলসে?" উত্তরটা না জানলে কোড দুবার লেখা হয়।

ধাপ ২

খোলসের ভেতরে কী কী ঢোকে

`templates/app_base.html` — মোট ১১ লাইন, আর ৪৪টি পর্দার কাঠামো তৈরি

```
{% extends "base.html" %}
{% block body %}
<div class="ims-shell">
  {% include "partials/sidebar.html" %}
  <div class="ims-main">
    {% include "partials/topbar.html" %}
    <div class="ims-content">
      {% include "partials/messages.html" %}
      {% include "partials/billing_banner.html" %}
      {% block content %}{% endblock %}
    </div>
  </div>
</div>
{% endblock %}
```

অংশ	দায়িত্ব
sidebar	ভূমিকা অনুযায়ী মেনু
topbar	শিরোনাম · workspace পিল · প্রোফাইল
messages	সেভ / এরর বার্তা
billing_banner	ট্রায়াল · read-only · সিট
`block content`	**একমাত্র** যা পর্দা বদলায়

নিয়ম: পর্দা লিখলে **শুধু** `content` **ব্লক** লেখা হয়। কেউ sidebar নিজে আঁকলে সেটা বাগের শুরু — দুবার রক্ষণাবেক্ষণ।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে একই ফাইল বানাতে — ১০ লাইনে। যে প্রজেক্টে প্রতিটি পেজে আলাদা মেনু লেখা, সেখানে নতুন মেনু আইটেম যোগ করতে ৩০টি ফাইল ছুঁতে হয়।

ধাপ ৩

ওয়্যারফ্রেম — বক্স আর মাপ, রং পরে

IMS-এর আসল মাপ: sidebar ২৬৪px · topbar ৫২px · content সর্বোচ্চ ১২৮০px · কার্ড রেডিয়াস ১৪px



মাপ আগে ঠিক করুন, রং পরে। রং আগে ঠিক করলে লেআউট বদলালে সব রং আবার বদলাতে হয়।

শিক্ষকের নোট: কাগজে ৫ মিনিটের কাজ দিন — শুধু বক্স আর সংখ্যা, কোনো রং নয়। যে দল মাপ লেখে না, তাদের ওয়্যারফ্রেম কোডিংয়ের সময় বদলে যাবে।

ধাপ ৪

১৫টি টোকেন — রং কোথাও হার্ডকোড নয়

`static/css/ims.css` — `:root`-এ ১৫টি ভেরিয়েবল, বাকি ১০০ লাইন শুধু সেগুলো ব্যবহার করে

শ্রেণি	টোকেন	কেন লাগে
রং (৯)	`primary` · `primary-dark` · `primary-soft`	ব্র্যান্ড বদলালে এখানেই
	`ink` · `ink-2` · `muted` · `line` · `bg` · `surface`	লেখা ও পৃষ্ঠতলের ধাপ
অবস্থা (৩)	`success` · `warning` · `danger`	একই অর্থে সব জায়গায় একই রং
আকৃতি (২)	`radius` · `radius-sm`	১৪px কার্ড, ১০px বাটন — মিশ্রণ নয়
গভীরতা (১)	`shadow`	দুই স্তরের ছায়া, একটাই ডেফিনিশন

টোকেন থাকলে: ১ লাইনে ব্র্যান্ড রং — `--ims-brand`।

টোকেন না থাকলে: ৭৫টি ফাইলে #4f46e5 খুঁজে বদলাতে হয় — একটা বাদ পড়বেই।

শিক্ষকের নোট: ছাত্রদের ১০টি রঙের নাম জিজ্ঞেস করুন — বেশিরভাগের উত্তর ভিন্ন হবে। বলুন: "আপনার নিজের প্রজেক্টে একই সবুজ কতবার লেখা আছে? grep করে দেখুন।"

ধাপ ৫

ভূমিকা অনুযায়ী পর্দা — এক ভিউ, তিন রূপ

`apps/reports/views.py` — `dashboard()` এক, কিন্তু টেমপ্লেট আলাদা

কে	কোন টেমপ্লেট	পর্দায় কী থাকে
STUDENT	`reports/dashboard_student`	উপস্থিতি % · প্রকাশিত পরীক্ষা · খোলা হোমওয়ার্ক · জমা দেওয়ার সংখ্যা
TEACHER / COORDINATOR / ORG_ADMIN	`reports/dashboard_staff`	৪টি KPI — শিক্ষার্থী · আজকের উপস্থিতি · অপ্রকাশিত পরীক্ষা · পর্যালোচনার অপেক্ষায়
তাঁদের মধ্যে ম্যানেজার	একই টেমপ্লেট	+ সদস্য সংখ্যা · অডিট লগ · উপস্থিতি নীতি
PLATFORM_ADMIN	`core/dashboard_platform`	মোট প্রতিষ্ঠান · ব্যবহারকারী · ইনভয়েস

আসল ফাঁদ: `core/` ফোল্ডারে ৪টি ড্যাশবোর্ড টেমপ্লেট আছে (`dashboard_teacher`, `_student`, `_coordinator`, `_institution`) — কোনো ভিউ সেগুলো রেন্ডার করে না। পুরোনো স্কেলিটন, এখনো ফাইলে পড়ে আছে। **মৃত টেমপ্লেট = বিভ্রান্তি:** নতুন ডেভেলপার ওটাই এডিট করে বাগ খুঁজবে।

শিক্ষকের নোট: এই লাইনটা বলুন: "কোড আছে মানে কোড চলে না।" নিজের প্রজেক্টে শেখার নিয়ম — যে ফাইল কোথাও ব্যবহৃত হয় না, সেটা মুছুন, নাহলে ভবিষ্যতে আপনি নিজেই ভুল ফাইল খুলবেন।

ধাপ ৬

একটি মেনু-তালিকা, তিনটি ব্যবহার

`apps/core/navigation.py` — ৫টি গ্রুপ · ২২টি আইটেম · ভূমিকা-ফিল্টার

```
NAV = [
  {"title": None, "items": [
     _item("Dashboard", "bi-speedometer2",
           "reports:dashboard",
           STAFF_ROLES + (Roles.STUDENT,))]},
  {"title": "Teaching", "items": [
     _item("Students", "bi-people",
           "students:list", STAFF_ROLES),
     _item("Attendance", "bi-calendar-check",
           "attendance:take", STAFF_ROLES), ...]},
  ...
]
def navigation_for(request):
    role = request.membership.role
    # ভূমিকা না মিললে আইটেম বাদ
    if role not in item["roles"]: continue
    url = reverse(item["url_name"], kwargs={"slug": slug})
    ...
```

গ্রুপ	আইটেম	কে
(শিরোনামহীন)	Dashboard	সবাই
Teaching	১০	STAFF
My learning	৩	STUDENT
Administration	৬	ORG_ADMIN
Account	২	সবাই

তিনটি ব্যবহার: **sidebar রেন্ডার** · **ড্যাশবোর্ড হাইলাইট** · **টেস্ট** ("এ ভূমিকা ওই লিংক দেখবে না")।

মেনু **দেখানো** নিয়ন্ত্রণ করে, **অনুমতি** নয়। সার্ভার প্রতিটি অনুরোধ আলাদা করে যাচাই করে — লিংক লুকানো মানে নিরাপত্তা নয় (ক্লাস ৯ দেখুন)।

শিক্ষকের নোট: শেষ লাইনটি জোর দিন — ছাত্ররা মনে করে লিংক লুকালেই কাজ শেষ। বলুন: "URL হাতে টাইপ করলে কী হয়? সেটাই আসল প্রশ্ন।"

ধাপ ৭

একই পর্দা, ভূমিকা অনুযায়ী ছোট ছোট পার্থক্য

পুরো পর্দা বদলানো ব্যয়বহুল — তাই টেমপ্লেটের ভেতরে শর্ত

শর্ত	কী বদলায়	কেন
`{% if nav_role == "ORG_ADMIN" %}`	ট্রায়াল ব্যানারে "প্ল্যান বাছুন" লিংক	শিক্ষক নবায়ন করতে পারেন না → লিংক দিলে ৪০৩
`{% if platform_console %}`	sidebar-এ Platform গ্রুপ	সাধারণ ব্যবহারকারীর কাছে অদৃশ্য
`{% if brand.logo %}`	লোগো, নাহলে আইকন	ব্র্যান্ডিং সবসময় পূর্ণ নয়
`{% if subscription.is_writable %}`	ব্যানারের ধরন	read-only অবস্থায় ভিন্ন বার্তা
`{% if e.is_published %}`	Published / Draft ব্যাজ	অপ্রকাশিত নম্বর শিক্ষার্থীর কাছে নেই

নিয়ম: শর্ত মানে **"না দেখানো"**, কখনো **"না করা"** নয়। প্রতিটি শর্তের সমান্তরাল সার্ভার-চেক থাকতেই হবে।

শিক্ষকের নোট: একটা উদাহরণ দিন: "ব্যানারে নবায়ন লিংক সবাইকে দিলে শিক্ষক ক্লিক করবেন, ৪০৩ পাবেন — আর ভাববেন সিস্টেম নষ্ট। শর্তটা ভদ্রতার জন্য, নিরাপত্তার জন্য নয়।"

ধাপ ৮

খালি অবস্থা — শূন্য মানে ভাঙা নয়

৩৩টি টেমপ্লেটে ৪১টি `ims-empty` · ১৬টি `{% empty %}` ব্লক

কোথায়	কী লেখা
classes	"No classes match this filter."
students	"No students enrolled yet."
attendance	"No attendance recorded this month."
exams	"No marks entered yet."
materials	"No materials uploaded yet."

১. আইকন

বোঝায় এটা তালিকা, এরর নয় — `bi-people` , `bi-award` ।

২. এক বাক্য

কী নেই, আর **কেন** নেই — "this month", "match this filter"।

৩. পরের ধাপ

সেরাটা: "Create an academic year first, then a term." — পথ দেখায়।

সাধারণ ভুল: শূন্য তালিকায় **সাদা স্ক্রিন**। ব্যবহারকারী ভাবেন সাইট আটকে গেছে — আসলে প্রশ্নটা ছিল "এখানে কিছু নেই, এখন কী করব?"

শিক্ষকের নোট: ক্লাসে ৩০ সেকেন্ডের কাজ দিন: নিজের প্রজেক্টের যে তালিকায় এখনো ডেটা নেই, তার জন্য এক বাক্য লিখুন। এটাই সবচেয়ে সস্তা উন্নতি।

ধাপ ৯

ফর্মের এক টুকরো — ৪৩টি ফর্ম, এক প্যাটার্ন

`templates/partials/field.html` — ৬ লাইন, প্রতিটি ফিল্ডের অভিন্ন চেহারা

```
<div class="mb-3">
  <label class="form-label">
    {{ field.label }}
    {% if field.field.required %}
      <span class="text-danger">*</span>
    {% endif %}</label>
  {{ field }}
  {% if field.help_text %}
    <div class="ims-hint">{{ field.help_text }}</div>
  {% endif %}
  {% for error in field.errors %}
    <div class="ims-error">{{ error }}</div>
  {% endfor %}
</div>
```

লেবেল + বাধ্যতামূলক

তারকা চিহ্ন সবসময় একই জায়গায় — চোখে পড়ে।

সহায়ক লেখা

`help_text` ফিল্ডের নিচে, ছোট ও ধূসর — প্লেসহোল্ডারে নয়।

এরর ফিল্ডের ঘরে

উপরের সারাংশে নয় — ব্যবহারকারী ঠিক কোন ঘর ঠিক করবেন দেখতে পান।

একই কারণে লগইন পাতার নিচে `ims-slice-note` বক্সে ডেমো লগইন লেখা থাকে — কাগজে লিখে দেওয়ার দরকার নেই।

শিক্ষকের নোট: বলুন: "একটা প্যাটার্ন বানান, তারপর প্রতিটি পেজে সেটাই include করুন। ২০টি ফর্ম হাতে সাজালে ২০টি ভিন্ন উচ্চতা, ভিন্ন এরর-রং — আর ব্যবহারকারী ভাববেন সাইটটা ভাঙা।"

ধাপ ১০

ব্র্যান্ডিং — এক প্রতিষ্ঠানের রং, সবার কোডে

`BrandingForm`-এ ৭টি ঘর · `base.html`-এ একটাই ভেরিয়েবল

```
:root {
  --ims-brand: {{ brand.color|default:"#4f46e5" }};
}
.btn-ims,
.ims-nav.is-active,
.ims-auth__submit { background: var(--ims-brand); }
.ims-sidebar {
  background: linear-gradient(180deg,
    color-mix(in srgb, var(--ims-brand) 22%, #0b1220),
    #0b1220);
}
a { color: var(--ims-brand); }
```

ঘর	কোথায় দেখা যায়
`display_name`	sidebar · লগইন শিরোনাম
`primary_color`	বাটন · সক্রিয় মেনু · sidebar
`tagline`	লগইনের বড় লেখা
`logo`	sidebar ও লগইন
`login_message`	লগইনের নির্দেশনা
`support_email` · `phone`	সহায়তার তথ্য

সেটিংস **ডেটাবেসে**, কোডে নয় — তাই একই কোডবেসে ১০টি প্রতিষ্ঠান, ১০টি রং।

ফাঁদ: ব্যবহারকারী নিজের পছন্দের রং বাছেন — হালকা হলুদে সাদা লেখা পড়া যায় না। সেটিংসে **অন্ধকার রঙের তালিকা**, আর প্রিভিউ কার্ড রাখতেই হয়।

শিক্ষকের নোট: ব্র্যান্ডিং পাতার "Preview" কার্ডটাই আসল শিক্ষা — সেভ করার আগেই ব্যবহারকারী ফলাফল দেখেন। বলুন: "যেখানে ব্যবহারকারী ভুল করতে পারেন, সেখানে আগেই ফলাফল দেখান।"

ধাপ ১১

বার্তা ও গার্ড — ফর্মের আগেই সতর্কবার্তা

দুই partial: `billing_banner` (৪ অবস্থা) · `cookie_guard` (১৫ লাইন JS)

অবস্থা	রং	ব্যবহারকারীকে যা বলা হয়
TRIAL	নীল	কত দিন বাকি · কোন প্ল্যান · "প্ল্যান বাছুন"
মেয়াদ শেষ (read-only)	কমলা	"আপনার ডেটা নিরাপদ ও এক্সপোর্টযোগ্য — নবায়ন করুন"
SUSPENDED	কমলা	কারণ · "ডেটা রাখা আছে, পড়া যায়"
সিট ৯০%+	কমলা	"৪৫০/৫০০ সিট ব্যবহৃত" · আপগ্রেড
কুকি বন্ধ	হলুদ	"এই ভিউতে লগইন ব্যর্থ হবে — সাধারণ ট্যাবে খুলুন"

বার্তার নিয়ম: কী হয়েছে → কেন → এখন কী করব। তিনটি ছাড়া বার্তা নয়।

গার্ড কেন: cookieless iframe-এ লগইন বারবার ব্যর্থ হয় — ব্যবহারকারী ৫ মিনিট টাইপ করার **আগেই** জানানো হয়।

শিক্ষকের নোট: "আপনার ডেটা নিরাপদ" কথাটা কেন লেখা — জিজ্ঞেস করুন। উত্তর: ভয় পাওয়া ব্যবহারকারী সাপোর্টে ফোন করে; আশ্বস্ত করা ব্যবহারকারী নবায়ন করে।

অনুশীলন

২০ মিনিট — নিজের প্রজেক্টের তিনটি খোলস

কাগজে, তারপর কোডে — এই ক্রমেই

১. খোলস আলাদা করুন (৫ মিনিট)

আপনার সিস্টেমে কত ধরনের পর্দা? (লগইন · অ্যাপ · পাবলিক · অ্যাডমিন) প্রতিটির জন্য **একটি** base ফাইল ঠিক করুন।

২. ওয়্যারফ্রেম আঁকুন (৭ মিনিট)

প্রধান পর্দার বক্স + মাপ (px)। রং নয়, ছবি নয় — শুধু আয়তক্ষেত্র আর সংখ্যা।

৩. তিনটি খালি অবস্থা লিখুন (৫ মিনিট)

যে তালিকা এখনো শূন্য, তার জন্য এক বাক্য — কী নেই, কেন নেই, এখন কী করব।

৪. টোকেন তালিকা (৩ মিনিট)

১০–১৫টি ভেরিয়েবল লিখুন — রং, মাপ, ছায়া। তারপর নিজের CSS-এ grep করুন কতগুলো হার্ডকোড রং আছে।

যাচাই: আপনার প্রজেক্টে একটা নতুন পর্দা যোগ করতে কয়টি ফাইল ছুঁতে হলো? ১ হলে ডিজাইন ঠিক, ৫ হলে খোলস ভাঙা।

শিক্ষকের নোট: শেষ প্রশ্নটাই মূল্যায়নের আসল মাপকাঠি — ফাইল সংখ্যা গুনে হাত তুলতে বলুন। যার উত্তর ৫, তার কাজ পরের সপ্তাহে হালকা।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে — ব্যাখ্যা বাড়াবেন না

- শিক্ষকের ড্যাশবোর্ড আর প্রতিষ্ঠান প্রধানের ড্যাশবোর্ড — এক টেমপ্লেটে হবে, নাকি দুটো? কোন তথ্য ভিন্ন?
- খালি অবস্থায় "কোনো ডেটা নেই" লেখা কি যথেষ্ট? ব্যবহারকারী এরপর কী চান?
- ব্র্যান্ড রং ব্যবহারকারী বদলাতে পারলে কোন রং নিষিদ্ধ করবেন, আর কীভাবে?
- মেনুতে লিংক লুকানো কি নিরাপত্তা? এক লাইনে যুক্তি দিন।

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তরেই ক্লাসের সারমর্ম — "না, লুকানো মানে শুধু দেখা যায় না; অনুমতি সার্ভারে।" এই উত্তর না এলে ক্লাস ৯-এর অংশ আবার বলুন।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
কাঠামো একবার, পর্দা বহুবার	৪টি খোলসে ৭৫টি টেমপ্লেট · `app_base.html` ১১ লাইন
মাপ আগে, রং পরে	wireframe: ২৬৪ / ৫২ / ১২৮০ / ১৪px · ১৫টি টোকেন
ভূমিকা = ভিন্ন পর্দা, ভিন্ন তথ্য	STUDENT · STAFF · PLATFORM — তিন ড্যাশবোর্ড, এক ভিউ
শূন্য অবস্থাও একটা পর্দা	৩৩ ফাইলে ৪১টি `ims-empty` · "এখন কী করব" লেখা
টেমপ্লেট শর্ত = ভদ্রতা, অনুমতি নয়	nav_role শর্ত + সার্ভার-চেক · লুকানো লিংক ≠ সুরক্ষা
মৃত কোড সরান	৪টি অপ্রয়োজনীয় ড্যাশবোর্ড টেমপ্লেট এখনো পড়ে আছে

শিক্ষকের নোট: শেষ সারিটা নিজের প্রজেক্টেও প্রযোজ্য — বলুন: "আজ রাতে ২০ মিনিট লাগবে, নিজের রিপোতে এমন মৃত টেমপ্লেট খুঁজে মুছে ফেলুন।"

১৩

পরের ক্লাস — "ডেটা কোথায় যায়"

DFD: Level 0 · Level 1 · ডেটা স্টোর · বাহ্যিক সত্তা · প্রসেসের নামকরণ

পর্ব ৩ · ডিজাইন · ক্লাস ১৩ / ৩৭

ডেটা কোথায় যায়

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.২ (DFD)
Level 0 · Level 1 · ৯টি স্টোর · ব্যাচ প্রসেস

৮৫টি রুট · ৩১টি টেবিল · ১১টি প্রসেস

শুরু করুন প্রশ্ন দিয়ে

একটি উপস্থিতি সেভ হলে ডেটা কত জায়গায় যায়?

হাত তুলুন: এক জায়গায়? তিন জায়গায়?

উপস্থিতির একটি সারি লেখা হয় ১টি টেবিলে,
কিন্তু তার ছাপ পড়ে অডিট লগে, রিপোর্টে, ড্যাশবোর্ডের সংখ্যায়।

DFD এই ছাপগুলো ছবিতে আঁকে — কে দেয়, কে নেয়, কোথায় জমা থাকে। কোড লেখার আগে এই ছবি থাকলে "এই ডেটা কোথা থেকে এলো?" প্রশ্নের উত্তর খুঁজতে তিন ঘণ্টা লাগে না।

শিক্ষকের নোট: উত্তর সাধারণত "এক জায়গায়" আসে। তারপর বলুন: "আজ আমরা সেই একটি লেখার পেছনের পাঁচটি পড়া বের করব।"

ধাপ ১

DFD-এর চারটি প্রতীক — এর বেশি লাগে না

চারটি আকৃতি, একটাই নিয়ম: প্রতিটি তীরের গায়ে লেখা থাকবে **কী যাচ্ছে**



নিয়ম: বাহ্যিক সত্তা সিস্টেমের বাইরে · প্রসেস কাজ করে · স্টোর শুধু রাখে, নিজে কিছু করে না · তীরের গায়ে সর্বদা নাম।

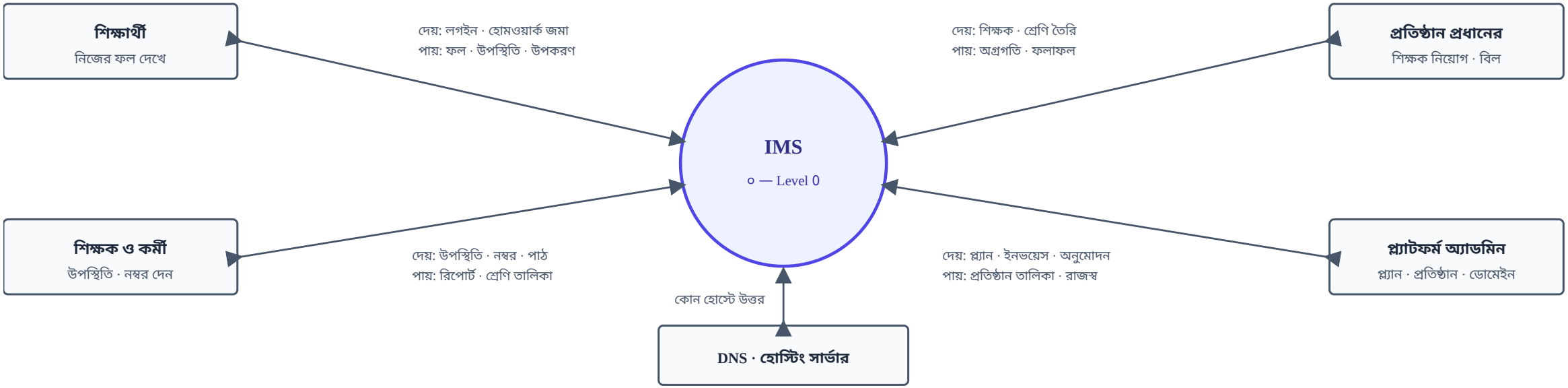
একটি গল্প বলতে চারটি প্রতীকই যথেষ্ট। রং, ছায়া, আইকন — ডায়াগ্রামের মান বাড়ায় না, **নামের স্পষ্টতা** বাড়ায়।

শিক্ষকের নোট: বোর্ডে চারটি আকৃতি আঁকুন, তারপর ছাত্রদের নিজের প্রজেক্টের একটি কাজকে এই চারটিতে ভাগ করতে বলুন। যার তীরে নাম নেই, তার DFD অর্ধেক।

ধাপ ২

Level 0 — পুরো সিস্টেম একটি বৃত্ত

পাঁচটি বাহ্যিক সত্তা · শূন্য ডেটা স্টোর · একটি প্রসেস



Level 0-তে **ডেটা স্টোর দেখানো হয় না** — কারণ বাইরের কেউ স্টোর দেখে না, সে সিস্টেমকে একটি কাজ হিসেবে দেখে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "প্ল্যাটফর্ম অ্যাডমিন কি বাহ্যিক সত্তা?" উত্তর: হ্যাঁ — সে IMS-এর ব্যবহারকারী নয়, IMS-কে চালায়। এই পার্থক্যটাই Level 0-এর মূল শিক্ষা।

ধাপ ৩

Level 1 — ১১টি অ্যাপ মানে ১১টি প্রসেস

কোনো সংখ্যা বানানো নয়: প্রতিটি অ্যাপের `urls.py` গুনে মোট ৮৫টি রুট

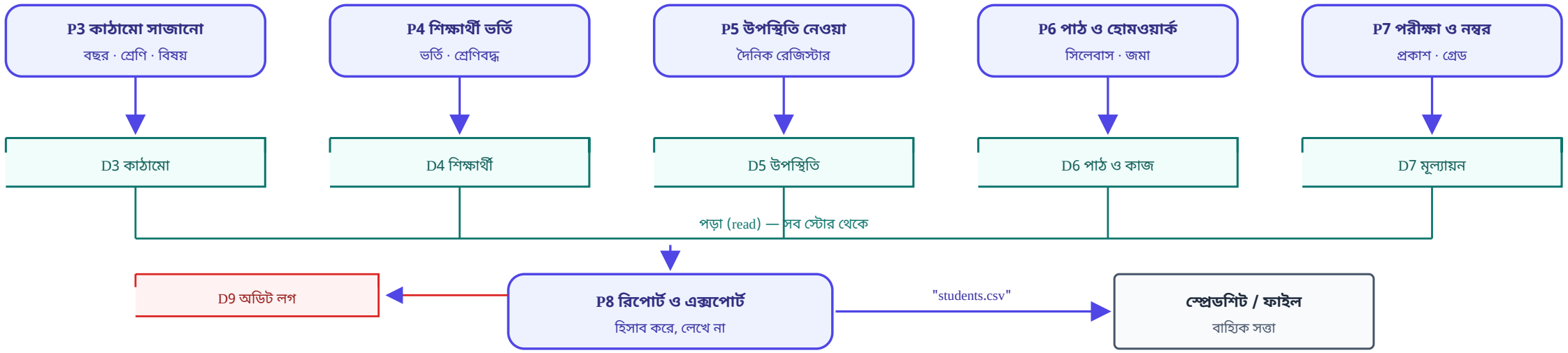
#	প্রসেস (ক্রিয়া)	অ্যাপ	রুট	যে স্টোরে লেখে
P1	লগইন ও সেশন চালানো	accounts	৯	D1 ব্যবহারকারী
P2	প্রতিষ্ঠান নির্ধারণ ও সদস্যতা	workspaces	২	D1 · D2
P3	একাডেমিক কাঠামো সাজানো	academics	১৩	D3 কাঠামো
P4	শিক্ষার্থী ভর্তি ও শ্রেণিবদ্ধ করা	students	৭	D4 শিক্ষার্থী
P5	উপস্থিতি নেওয়া ও সংশোধন	attendance	৪	D5 উপস্থিতি
P6	পাঠ, হোমওয়ার্ক ও উপকরণ দেওয়া	lessons	১৩	D6 পাঠ
P7	পরীক্ষা তৈরি ও নম্বর প্রকাশ	assessments	৯	D7 মূল্যায়ন
P8	রিপোর্ট তৈরি ও ফাইল বের করা	reports	৭	পড়ে D3–D7 · লেখে D9
P9	সাবস্ক্রিপশন ও ইনভয়েস	billing	১৮	D8 বাণিজ্যিক
P10	প্ল্যাটফর্ম প্রশাসন	core · views_platform	১ + ট্যাব	D8 · D2
P11	অডিট লেখা (সবাই ডাকে)	audit	২	D9 অডিট

শিক্ষকের নোট: বলুন: "Level 1 বানাতে নতুন কিছু ভাবতে হয় না — আপনার ফোল্ডার তালিকাই প্রসেসের তালিকা।" এতে DFD লেখা সহজ হয়ে যায়, আর ভুল কমে।

ধাপ ৪

Level 1 ছবিতে — লেখা আর পড়া আলাদা

প্রতিটি প্রসেস নিজের স্টোরে লেখে; রিপোর্ট সবার স্টোর পড়ে — একটিতেও লেখে না



P8 একটি স্টোরেও লেখে না — কিন্তু অডিটের জন্য ডাকে। তাই D9-এ তার লেখা দুটি: "students.csv" আর "attendance.save"।

শিক্ষকের নোট: তীরের দিকটাই এখানে আসল শিক্ষা — একই প্রসেস সরবরাহ করে, আবার পড়ে; কিন্তু কোনো প্রসেস কখনো একই স্টোরে দুই দিকে তীর আঁকে না।

ধাপ ৫

৩১টি টেবিল, কিন্তু ৯টি ডেটা স্টোর

DFD-তে টেবিল গোনা হয় না — **একসাথে রাখা দরকার** এমন জিনিস গোনা হয়

স্টোর	টেবিল	ভেতরে কী
D1 ব্যবহারকারী ও সদস্যতা	২	User · Membership (ভূমিকা, সক্রিয়তা)
D2 প্রতিষ্ঠান ও ব্র্যান্ডিং	২	Workspace · WorkspaceBranding (ডোমেইন, রং)
D3 একাডেমিক কাঠামো	৭	বছর · টার্ম · বিভাগ · শ্রেণি · বিষয় · শিক্ষক-বণ্টন · কোঅর্ডিনেটর স্কোপ
D4 শিক্ষার্থী ও ভর্তি	২	StudentProfile · Enrollment
D5 উপস্থিতি	২	AttendanceRecord · AttendancePolicy
D6 পাঠ ও কাজ	৭	Syllabus · Chapter · Topic · LessonLog · Homework · Submission · Material
D7 মূল্যায়ন	৪	GradeScale · GradeBand · Exam · Mark
D8 বাণিজ্যিক	৪	Plan · Subscription · Invoice · SignupRequest
D9 অডিট	১	AuditLog — ২৫টি ভিন্ন action

ডেটাবেসের বাইরেও দুটি স্টোর: **সেশন স্টোর** (কোন প্রতিষ্ঠানে আছেন) আর **মিডিয়া ফাইল** (উপকরণ, লোগো)। DFD-তে এগুলোও স্টোর — কারণ তারা টিকে থাকে।

শিক্ষকের নোট: ৯ বনাম ৩১ — এই পার্থক্যটা বলুন: "৩১ হলো আলমারির ড্রয়ার, ৯ হলো আলমারি। DFD-তে আলমারি আঁকি, ড্রয়ার নয় — নাহলে ছবিটা পড়া যায় না।"

ধাপ ৬

নামকরণ — DFD-এর সবচেয়ে সস্তা গুণ

ভুল নাম থাকলে ছবিটা ভুল হয়ে যায়, আর কেউ ধরতে পারে না

কী	ভুল	ঠিক
প্রসেস	"শিক্ষকের কাজ" · "Attendance"	"উপস্থিতি রেকর্ড করা" — **ক্রিয়া + বিশেষ্য**
স্টোর	"উপস্থিতি নেওয়া"	"D5 উপস্থিতি" — **বিশেষ্য**, সংখ্যায়িত
সত্তা	"ড্যাশবোর্ড" · "ডেটাবেস"	"শিক্ষক" · "প্ল্যাটফর্ম অ্যাডমিন" — **সিস্টেমের বাইরের মানুষ**
তীর	"ডেটা" · "তথ্য"	"উপস্থিতির সারি" · "নম্বর প্রকাশের অনুরোধ"

খারাপ তীর: "ডেটা" — পড়ে কেউ বলতে পারে না কী যাচ্ছে, তাই ভুল জায়গায় লেখা ধরাও পড়ে না।

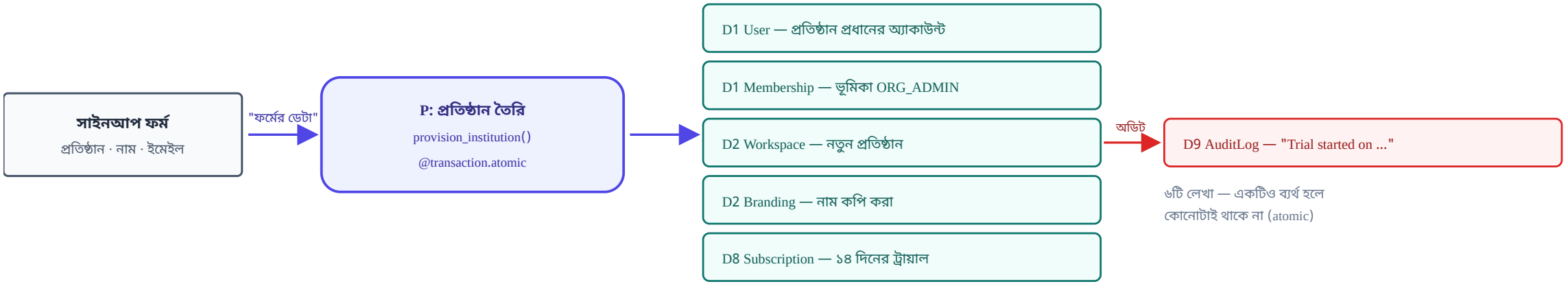
ভালো তীর: "উপস্থিতির সারি (শিক্ষার্থী, তারিখ, অবস্থা)" — নিজেই একটা স্কিমার খসড়া।

শিক্ষকের নোট: শ্রেণিতে একটি নিয়ম চালু করুন: তীরের গায়ে "ডেটা" লেখা পেলে খাতা ফেরত। এক সপ্তাহে অভ্যাস হয়ে যাবে।

ধাপ ৭

একটি সেভ, ছয়টি স্টোরে লেখা

`provision_institution()` — পাবলিক সাইনআপ ফর্ম থেকে প্রতিষ্ঠান তৈরি



DFD-তে একটি প্রসেস একাধিক স্টোরে লিখতে পারে — কিন্তু **সব লেখা একসাথে সফল বা একসাথে ব্যর্থ**। এটাই ডেটা-অখণ্ডতার নিয়ম, আর কোডে সেটি `@transaction.atomic`।

শিক্ষকের নোট: বলুন: "যদি পঞ্চম লেখাটা ফেল করে, প্রথম চারটে থেকে গেলে কী হবে? ব্যবহারকারী তৈরি, প্রতিষ্ঠান নেই — অর্ধেক মানুষ।" এজন্যই transaction।

ধাপ ৮

একটি ডিজাইন-সিদ্ধান্ত: তাৎক্ষণিক, নাকি অনুমোদন?

একই DFD, দুই রকম তীর — পার্থক্য শুধু একটি প্রসেসে

প্রশ্ন	তাৎক্ষণিক (IMS যা করছে)	অনুমোদনের পর
ব্যবহারকারীর অভিজ্ঞতা	ফর্ম শেষ → সোজা ড্যাশবোর্ড	ফর্ম শেষ → "অপেক্ষা করুন" পর্দা
ট্রেইল	`SignupRequest` তৈরি হয়, অবস্থা **PROVISIONED**	তৈরি হয়, অবস্থা **NEW → অনুমোদিত**
স্প্যাম	ঝুঁকি বেশি → পরে **SPAM/REJECTED** মার্ক	ঝুঁকি কম, দেরি বেশি
কে থামাতে পারে	প্ল্যাটফর্ম অ্যাডমিন, পরে (suspend)	প্ল্যাটফর্ম অ্যাডমিন, আগেই

যেটাই বাছুন, **লিখে রাখুন কেন**। IMS বেছে নিয়েছে "আগে ঢোকা, পরে থামানো" — কারণ ট্রায়ালের মূল্য হলো শিক্ষক পাঁচ মিনিটে ভেতরে ঢুকতে পারেন। ঝুঁকির প্রতিকার DFD-তেই দেখা যায়: `SignupRequest` নামের একটি স্টোর আছে যেখানে IP ঠিকানা পর্যন্ত জমা থাকে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "আপনার প্রজেক্টে কোনটা?" তারপর বলুন: "উত্তরটা যেটাই হোক, DFD-তে সেই পথটাই তীর হয়ে আঁকতে হবে — নয়তো ছবি আর কোড আলাদা কথা বলবে।"

ধাপ ৯

প্রসেসের শেষ প্রান্ত: ফাইল

৩টি CSV এক্সপোর্ট — `reports/urls.py`-এ দেখা যায়, বাইরের সত্তা "স্প্রেডশিট"

```
def _csv_response(filename, header, rows):
    resp = HttpResponse(
        content_type="text/csv; charset=utf-8")
    resp["Content-Disposition"] = (
        f'attachment; filename="{filename}"')
    resp.write("\ufeff")   # BOM: Excel-এ বাংলা ঠিক দেখায়
    writer = csv.writer(resp)
    writer.writerow(header)
    for row in rows:
        writer.writerow(row)
    return resp
```

ফাইল	কোন স্টোর পড়ে
`students.csv`	D4 শিক্ষার্থী
`attendance-*.csv`	D5 উপস্থিতি
`results-*.csv`	D7 মূল্যায়ন

নিয়ম: ফাইল বের করা একটি **পড়ার** কাজ — কোনো স্টোরে লেখা হয় না, শুধু অডিটে ছাপ পড়ে।

ছাত্র `results.csv` চাইলে ৪০৩ — রিপোর্টও ভূমিকা মেনে চলে।

শিক্ষকের নোট: BOM লাইনটা দেখিয়ে বলুন: "এই একটি অক্ষর না থাকলে Excel-এ বাংলা অক্ষর ভাঙা দেখায়। DFD-তে এটা দেখা যায় না — কিন্তু ব্যবহারকারীর কাছে এটাই পুরো সিস্টেম।"

ধাপ ১০

ব্যাচ প্রসেস — যেখানে কোনো ব্যবহারকারী নেই

৪টি ম্যানেজমেন্ট কমান্ড: `seed_demo` · `seed_plans` · `billing_report` · `sync_domains`

কমান্ড	কোন স্টোরে	কী কাজ
`seed_plans`	D8 লেখে	ডিফল্ট প্ল্যান তৈরি — খালি ডেটাবেসে সিস্টেম চলে
`seed_demo`	D1–D7 লেখে	ডেমো প্রতিষ্ঠান, শিক্ষক, ছাত্র, উপস্থিতি
`billing_report`	D8 পড়ে	আয়ের সারসংক্ষেপ ছাপে
`sync_domains`	D2 পড়ে	ভেরিফায়েড ডোমেইনের CNAME ও `ALLOWED_HOSTS` লাইন ছাপে

DFD-তে ব্যাচ প্রসেস: যে প্রসেস ব্যবহারকারীর অনুরোধ ছাড়াই চলে, তাকেও আঁকতে হয় — নাহলে "এই রেকর্ড কে বানাল?" প্রশ্নের উত্তর ছবিতে থাকে না।

DNS বাইরের সত্তা: `sync_domains` শুধু **কী করতে হবে** ছাপে — DNS বদলায় না। অ্যাপ নিজে নিজের ডোমেইন বানাতে পারে না।

শিক্ষকের নোট: এখানে সততার পাঠ: "যা করতে পারে না, তা কোডে লেখা নেই। স্কুলকে CNAME বানাতে হবে — আমাদের কাজ লাইনটা ছাপা, ভান করা নয়।"

সতর্কতা

DFD-তে ছয়টি সাধারণ ভুল

পরীক্ষায় নম্বর কাটে, প্রজেক্টে বিভ্রান্তি ছড়ায়

ভুল	কারণ ও সমাধান
স্টোর → স্টোর সরাসরি তীর	টেবিল নিজে কিছু করে না। মাঝখানে প্রসেস বসান
Level 0-তে ডেটা স্টোর	Level 0 হলো বাইরের চোখের ছবি — ভেতরের আলমারি ওখানে নেই
Level 1 ও 2 এক পাতায়	এক পাতা, এক স্তর। নাহলে ছবি পড়া যায় না
প্রসেসের নামে মানুষ ("শিক্ষক যা করেন")	প্রসেস সবসময় **ক্রিয়া**: "উপস্থিতি রেকর্ড করা"
নামহীন তীর	"ডেটা" লেখাও নাম নয় — কী ডেটা, কার থেকে কার
অডিট স্টোর ভুলে যাওয়া	২৫টি action লেখা হয় — D9 বাদ দিলে ছবি অসম্পূর্ণ

সবচেয়ে বড় ভুল: **ছবি একবার আঁকা, তারপর কোড বদলালে ছবি না বদলানো।** DFD কোডের সাথে বাঁচে — বদলালে আপডেট করুন, নয়তো ছবিটা মিথ্যা বলে।

শিক্ষকের নোট: শেষ লাইনটা নিজের ছাত্রজীবনের উদাহরণ দিয়ে বলুন — "প্রথম প্রজেক্টে আমার ডায়াগ্রাম আর কোড দুই রকম হয়ে গিয়েছিল, ভাইভায় সেটাই ধরা পড়ল।"

অনুশীলন

২০ মিনিট — নিজের সিস্টেমের Level 0 আর Level 1

কাগজ就可以了 — হাতের আঁকাই যথেষ্ট

১. বাহ্যিক সত্তা লিখুন (৪ মিনিট)

কে সিস্টেমের বাইরের মানুষ? (৪–৬ জন) প্রত্যেকে কী দেয়, কী নেয় — দুই কলামে।

২. Level 0 আঁকুন (৪ মিনিট)

একটি বৃত্ত, চারপাশে সত্তা, প্রতিটি তীরে নাম। **কোনো স্টোর নয়।**

৩. Level 1 ভাঙুন (৭ মিনিট)

আপনার অ্যাপ/মডিউল = প্রসেস। প্রতিটির পাশে লিখুন কোন স্টোরে **লেখে** আর কোনটা **পড়ে**।

৪. যাচাই (৫ মিনিট)

আপনার মডেল ফাইল খুলুন — DFD-র প্রতিটি স্টোরের টেবিল আছে? টেবিল আছে কিন্তু স্টোর নেই — কোথায়?

যাচাইয়ের প্রশ্ন: DFD দেখে একজন নতুন ডেভেলপার বলতে পারবে "উপস্থিতির ডেটা কোথা থেকে আসে?" — না পারলে ছবি অসম্পূর্ণ।

শিক্ষকের নোট: চতুর্থ ধাপটাই আসল কাজ — ছাত্ররা দেখবে তাদের কোডে এমন টেবিল আছে যা DFD-তে নেই (বা উল্টো)। সেটাই পরের ক্লাসের (ERD) সূচনা।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- প্ল্যাটফর্ম অ্যাডমিন Level 0-তে বাহ্যিক সত্তা, কিন্তু সে তো আমাদেরই একজন — কেন বাইরে?
- অডিট লগ কি "স্টোর" না "প্রসেস"? এক লাইনে যুক্তি দিন।
- সাইনআপ তাৎক্ষণিক না অনুমোদনসাপেক্ষ — আপনার প্রজেক্টে কোনটা, আর কেন?
- DFD-তে যা দেখানো হয় না, কোডে তা থাকলে কী সমস্যা? উদাহরণ দিন।

শিক্ষকের নোট: চতুর্থ প্রশ্নে উদাহরণ আসবে "ব্যাকগ্রাউন্ড জব" বা "ক্যাশ"। বলুন: "যা ছবিতে নেই, তা কেউ রক্ষণাবেক্ষণও করে না — ছবিটাই সিস্টেমের স্মৃতি।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
Level 0 = বাইরের চোখ — এক প্রসেস, স্টোর নেই	৫টি সত্তা · ১টি বৃত্ত · ০ স্টোর
Level 1 = আপনার ফোল্ডার তালিকা	১১ অ্যাপ · ১১ প্রসেস · ৮৫ রুট
স্টোর গোনা হয় দরকারে, টেবিলে নয়	৩১ টেবিল → ৯ স্টোর + সেশন + মিডিয়া
প্রতিটি সেভ একাধিক স্টোর ছুঁতে পারে	`provision_institution()` — ৬টি লেখা, একটি transaction
ফাইল বের করা = পড়া, লেখা নয়	৩টি CSV · BOM সহ · ছাত্রের জন্য ৪০৩
ব্যাচ প্রসেস আর বাইরের সিস্টেমও ছবিতে থাকে	৪টি কমান্ড · DNS CNAME · `ALLOWED_HOSTS`

শিক্ষকের নোট: তৃতীয় সারিটা বোর্ডে লিখুন — "৩১ → ৯"। এই একটি সংখ্যাই DFD আর ERD-এর পার্থক্য বোঝায়।

১৪

পরের ক্লাস — "কোন ডেটা কার সাথে থাকে"

ERD: ৩১টি মডেল · সম্পর্কের ধরন · কী-এর পছন্দ · স্কোপ কলাম কেন প্রতিটি টেবিলে

পর্ব ৩ · ডিজাইন · ক্লাস ১৪ / ৩৭

কোন ডেটা কার সাথে থাকে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.৩ (ডেটাবেস ডিজাইন)

ERD: ৩১ টেবিল · ৭৫ সম্পর্ক · কার্ডিনালিটি

৭০টি FK · ৪টি 1:1 · ১টি M:N

শুরু করুন প্রশ্ন দিয়ে

একজন শিক্ষার্থী চারটি শ্রেণিতে ছিল — কোন টেবিলে লেখা থাকে?

হাত তুলুন: শিক্ষার্থীর টেবিলে? নাকি আলাদা টেবিলে?

৩১টি টেবিল, ৭৫টি সম্পর্ক। সম্পর্কটা না বুঝে কোড লিখলে শ্রেণি বদলানোর সময় পুরোনো ইতিহাস হারিয়ে যায়।

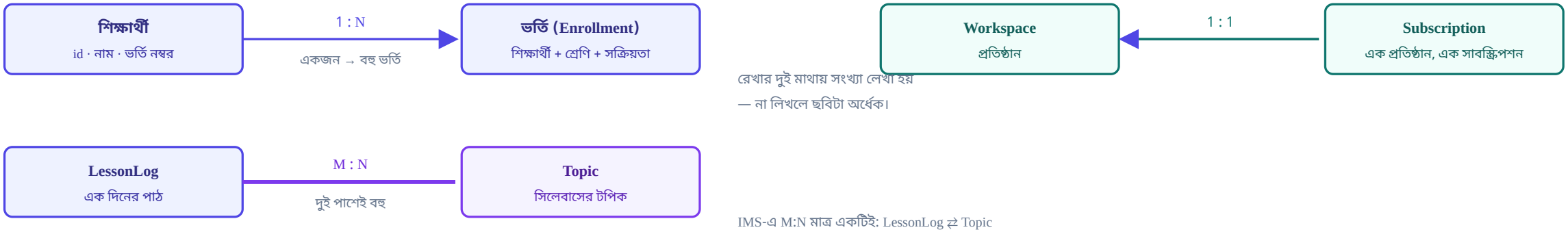
IMS-এ উত্তরটি হলো `Enrollment` — শিক্ষার্থী আর শ্রেণির মাঝখানের টেবিল। আজকের ক্লাসে কেন উত্তরটা শিক্ষার্থীর টেবিলে **হতে পারে না**, সেটাই মূল আলোচনা।

শিক্ষকের নোট: বেশিরভাগের উত্তর আসবে "শিক্ষার্থীর টেবিলে class_id"। বলুন: "তাহলে গত বছরের শ্রেণি কোথায় থাকল? মুছে গেল?" — এই একটি প্রশ্নেই আজকের ক্লাস দাঁড়িয়ে যায়।

ধাপ ১

ERD পড়ার তিনটি চিহ্ন

বাক্স = টেবিল · রেখা = সম্পর্ক · রেখার মাথার লেখা = কতজন



কার্ডিনালিটি পড়ার নিয়ম: **"এক X-এর অধীনে কতজন Y?"** — উত্তর ১ হলে 1:1, বহু হলে 1:N, আর দুই দিকেই বহু হলে M:N।

শিক্ষকের নোট: ছাত্রদের দিয়ে প্রশ্নটাই উচ্চারণ করান: "এক শ্রেণিতে কতজন ছাত্র?" (বহু) "একজন ছাত্র কত শ্রেণিতে?" (বহু, সময়ের সাথে) — তাই মাঝখানে টেবিল।

ধাপ ২

৭৫টি সম্পর্কের হিসাব

`grep "ForeignKey\|OneToOneField\|ManyToManyField" apps/*/models.py` — হাতে গোনা

অ্যাপ	FK	1:1	M:N	মোট
lessons	২১	—	১	**২২**
academics	১৭	—	—	**১৭**
assessments	১০	—	—	**১০**
attendance	৬	১	—	**৭**
billing	৬	১	—	**৭**
students	৫	১	—	**৬**
workspaces	৩	১	—	**৪**
audit	২	—	—	**২**

মোট **৭৫**: ৭০টি FK · ৪টি 1:1 · ১টি M:N।
৩১টি টেবিলের মধ্যে **২৩টি** সরাসরি প্রতিষ্ঠান-স্কোপড।

lessons-এ সবচেয়ে বেশি সম্পর্ক (২২) — কারণ এক পাঠে টপিক, হোমওয়ার্ক, জমা, উপকরণ, শ্রেণি, বিষয়, শিক্ষক — সাতটি জিনিস একসাথে জড়ায়। ডিজাইনে সবচেয়ে কঠিন অ্যাপও এটাই।

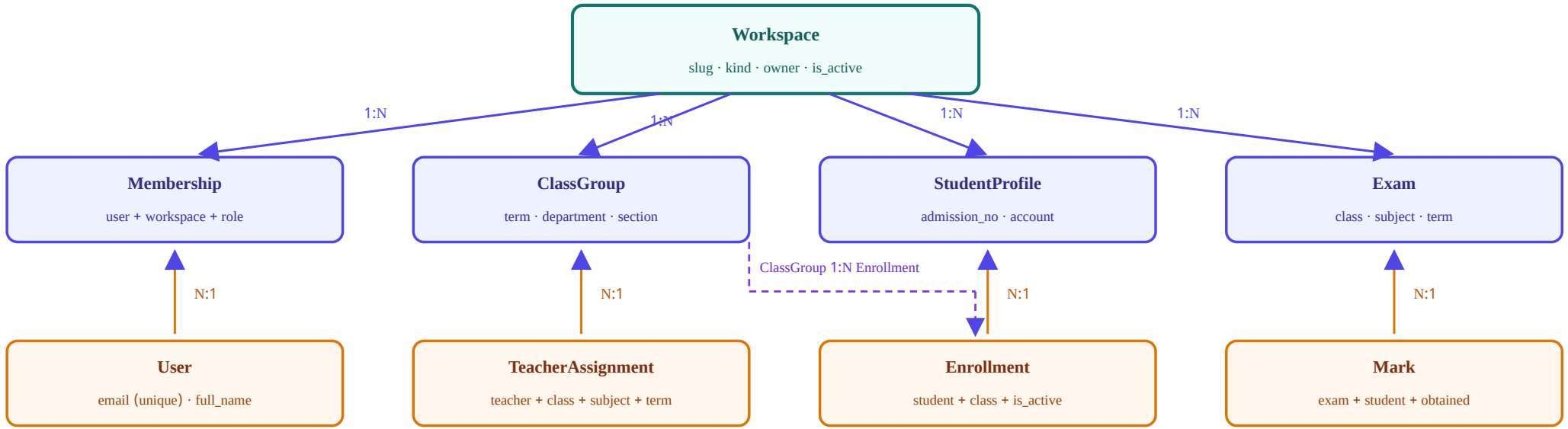
অডিটে মাত্র ২টি সম্পর্ক — তবু প্রতিটি প্রসেস সেখানে লেখে। সম্পর্ক কম মানে গুরুত্ব কম নয়।

শিক্ষকের নোট: বোর্ডে ৭৫ লিখুন, নিচে ২৩। বলুন: "৭৫ সম্পর্কের মধ্যে ২৩টি প্রতিষ্ঠানকে ধরে রাখে — বাকিগুলো ভেতরের গল্প।"

ধাপ ৩

ERD — প্রথম ভাগ: প্রতিষ্ঠান, মানুষ ও ভূমিকা

চারটি টেবিলেই প্রতিষ্ঠানের সীমা তৈরি হয়: Workspace · Membership · ClassGroup · Enrollment



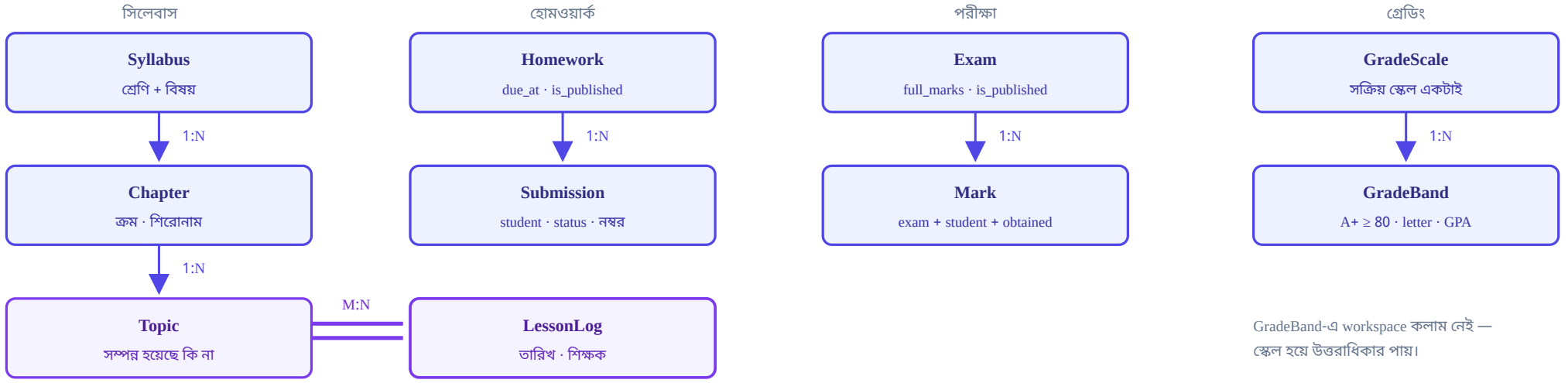
লক্ষ্য করুন: **User** আর **Workspace** সরাসরি জুড়তে হয় না — মাঝখানে Membership, কারণ এক ব্যবহারকারী বহু প্রতিষ্ঠানে ভিন্ন ভূমিকায় থাকতে পারে।

শিক্ষকের নোট: Membership-এর জায়গাটা আঙুল দিয়ে দেখান: "ভূমিকা কোথায় থাকবে? User-এ রাখলে এক ব্যবহারকারীর একটাই ভূমিকা হতো — শিক্ষক আবার অভিভাবক হতে পারতেন না।"

ধাপ ৪

ERD — দ্বিতীয় ভাগ: পাঠ, কাজ ও মূল্যায়ন

তিনটি চেইন: সিলেবাস · হোমওয়ার্ক · পরীক্ষা — আর একটাই M:N



তিনটি চেইনই **1:N** — কারণ পিতৃ-টেবিলের একটি সারির অধীনে বহু সন্তান। M:N একমাত্র সিলেবাসে, কারণ এক পাঠে একাধিক টপিক, আর এক টপিক বহু পাঠে।

শিক্ষকের নোট: ছাত্রদের বলুন প্রতিটি চেইনে "পিতা কে, সন্তান কে" মুখে বলতে — মুখে বলতে না পারলে ERD মুখস্থ, বোঝা নয়।

ধাপ ৫

চারটি 1:1 — কেন আলাদা টেবিল

1:1 মানে "একটির বেশি হতে পারে না" — সেটাই ডিজাইনের যুক্তি

সম্পর্ক	কেন একটির বেশি নয়	আলাদা টেবিল না হলে কী হতো
Workspace → Subscription	এক প্রতিষ্ঠানের একটাই চুক্তি	সাবস্ক্রিপশন ফিল্ড ৭টি প্রতিষ্ঠান-টেবিলে ঢুকে যেত
Workspace → AttendancePolicy	একটি উপস্থিতি নীতি	নীতি ছাড়া প্রতিষ্ঠান চলত, NULL-এ ভরা কলাম
Workspace → WorkspaceBranding	এক সেট ব্র্যান্ডিং	ব্র্যান্ডিং এডিট করতে প্রতিষ্ঠান-টেবিলে হাত দিতে হতো
User → StudentProfile	একজন ব্যবহারকারী এক শিক্ষার্থী	লগইন ছাড়া ছাত্র থাকত না — অথচ অ্যাকাউন্ট ঐচ্ছিক

সাধারণ নিয়ম: 1:1 ব্যবহার করি যখন তথ্য **ঐচ্ছিক**, **ভিন্ন দায়িত্বে**, বা **বড়** (ফাইল/JSON)।

ফাঁদ: 1:1-এ ডেটা দুই টেবিলে ভাগ হলে প্রতিটি কুয়েরিতে join লাগে — অকারণ ভাগ করলে সব ধীর ও জটিল হয়ে যায়।

শিক্ষকের নোট: চতুর্থ সারিটি জোর দিন: "StudentProfile.account ঐচ্ছিক — কারণ ছোট প্রতিষ্ঠানে সব ছাত্রের লগইন থাকে না। ERD-তে ঐচ্ছিকতা দেখা যায় NULL দিয়ে।"

ধাপ ৬

M:N একটাই — কিন্তু যৌগিক টেবিল অনেকগুলো

`LessonLog.topics` Django নিজেই মাঝের টেবিল বানায়; বাকি জায়গায় হাতে বানানো হয়েছে

```
class LessonLog(TimeStamped):
    topics = models.ManyToManyField(
        Topic, blank=True, related_name="lesson_logs")

class AttendanceRecord(TimeStamped):
    class Meta:
        constraints = [
          models.UniqueConstraint(
            fields=["student", "class_group",
                    "subject", "date"],
            name="uniq_attendance_student_class_"
                 "subject_date"),
        ]

class Mark(TimeStamped):
    class Meta:
        constraints = [
          models.UniqueConstraint(
            fields=["exam", "student"],
            name="uniq_mark_exam_student"),
        ]
```

Django-র M:N কখন যথেষ্ট

মাঝের টেবিলে **অতিরিক্ত তথ্য নেই** — শুধু "কোন পাঠে কোন টপিক"।

নিজের টেবিল কখন লাগে

যখন সম্পর্কের নিজের গুণ আছে — কে রেকর্ড করল, কখন, কোন অবস্থা। AttendanceRecord-এ ৮টি নিজের ফিল্ড।

ট্রিক: যৌগিক unique

M:N টেবিলে ডুপ্লিকেট আটকাতে ৪টি কলামের unique — একই দিনে একই ছাত্রের একই ক্লাসে দ্বিতীয় সারি অসম্ভব।

ফাঁদ: M:N ব্যবহার করলে **কে কখন যোগ করল** তথ্যটি হারায়। উপস্থিতি, নম্বর, জমা — এগুলো কখনোই "শুধু সম্পর্ক" নয়, এগুলো **ঘটনা**।

শিক্ষকের নোট: বলুন: "যদি প্রশ্নের উত্তর দিতে হয় 'কে করল, কখন করল' — তাহলে M:N নয়, নিজের টেবিল।" এই এক লাইনেই ছাত্রদের অনেক ভুল বাঁচবে।

ধাপ ৭

কী (Primary Key) — কখন অটো, কখন প্রাকৃতিক

৩১টি টেবিলে অটো `id`; কিন্তু ৬টি জায়গায় নিজস্ব ইউনিক কী

কী	ধরন	কেন এটা
`id` (সব টেবিলে)	অটো ইনক্রিমেন্ট	কখনো বদলায় না, ছোট, ইনডেক্সে হালকা
`User.email`	প্রাকৃতিক · unique	লগইনের পরিচয় — `USERNAME_FIELD`
`Workspace.slug`	প্রাকৃতিক · unique	URL-এ দেখা যায়: `/w/springfield/`
`StudentProfile.admission_no`	যৌগিক unique `(workspace, admission_no)`	প্রতিষ্ঠানভেদে একই নম্বর চলতে পারে
`Invoice.number`	প্রাকৃতিক · unique	হিসাবরক্ষণের নথি — পুনরায় ব্যবহার নিষিদ্ধ
`Plan.code`	প্রাকৃতিক · slug unique	কোডে উল্লেখযোগ্য: `starter`

নিয়ম: **PK অদৃশ্য থাকুক** (id), **মানুষের দেখা নম্বরটা unique কলাম হোক** (admission_no)।

ভুল: `admission_no`-কে PK বানানো — প্রতিষ্ঠান বদলালে বা নম্বর সংশোধন হলে পুরো সম্পর্ক ভাঙে।

শিক্ষকের নোট: জিজ্ঞেস করুন: "ভর্তি নম্বর ভুল ছিল, এখন ঠিক করবেন — PK হলে কী হবে?" উত্তর: সব foreign key ভেঙে যাবে। এজন্যই অটো id।

ধাপ ৮

ইউনিক — ৬টি পুরোনো, ৮টি নতুন

`unique_together` বনাম `UniqueConstraint` — দ্বিতীয়টি শর্ত দিতে পারে

unique_together (৬)	কীসের উপর
AcademicYear	workspace + name
AcademicTerm	workspace + year + name
Department	workspace + name
ClassGroup	workspace + term + name + section
Subject	workspace + name
TeacherAssignment	teacher + class + subject + term

UniqueConstraint (৮)	শর্ত
uniq_active_term_per_workspace	একটাই সক্রিয় টার্ম
uniq_active_grade_scale_per_workspace	একটাই সক্রিয় স্কেল
uniq_active_enrollment_student_class	একটাই সক্রিয় ভর্তি
unique_custom_domain	খালি ডোমেইন ছাড়া
uniq_membership_user_workspace	শর্ত নেই
uniq_mark_exam_student	শর্ত নেই
uniq_attendance_...date	শর্ত নেই
uniq_submission_homework_student	শর্ত নেই

প্রথম ৪টি **শর্তসাপেক্ষ (partial)** — শুধু "সক্রিয়" সারিগুলোর মধ্যে ইউনিক।

পুরোনো বছরগুলোর ইতিহাস রাখতে হবে, তাই `(workspace, year, name)` সাধারণ unique; কিন্তু একসাথে দুইটি **সক্রিয়** টার্ম চলা যাবে না — সেটি partial। বিস্তারিত ক্লাস ১৫-এ।

শিক্ষকের নোট: পার্থক্যটা এক বাক্যে: "সাধারণ unique বলে 'সবসময় একটাই'; partial unique বলে 'একটাই সক্রিয়, বাকিগুলো ইতিহাস'।"

ধাপ ৯

স্কোপ কলাম — ২৩টিতে সরাসরি, ৮টিতে উত্তরাধিকারে

`workspace` কলাম আছে কি না — প্রতিটি কুয়েরির নিরাপত্তা এর উপর নির্ভর করে

২৩টি টেবিলে সরাসরি `workspace`:
কাঠামো · ছাত্র · উপস্থিতি · পাঠ · পরীক্ষা · অডিট · সাবস্ক্রিপশন...

৮টিতে নেই: User · Workspace · Plan · GradeBand · Chapter · Topic · Mark · Submission

টেবিল	স্কোপ কোথা থেকে আসে
Chapter · Topic	`syllabus.workspace` — দুই ধাপ
Mark	`exam.workspace` — এক ধাপ
Submission	`homework.workspace` — এক ধাপ
GradeBand	`scale.workspace` — এক ধাপ
Plan · User	প্ল্যাটফর্ম-স্তরের, স্কোপ নেই
Workspace	নিজেই স্কোপ

ঝুঁকি: `Mark.objects.all()` লিখলে প্রতিষ্ঠান আলাদা হয় না — join করে ফিল্টার করতেই হবে।

তাই IMS-এ `scoping.py` নামের একটাই জায়গা — প্রতিটি তালিকা-ভিউ নিজে ফিল্টার লেখে না। স্কোপ ও কোয়েরি ক্লাস ১৫ ও পর্ব ৪-এ।

শিক্ষকের নোট: বলুন: "কলাম না থাকলে নিরাপত্তা হারায় না — কিন্তু ভুলে যাওয়ার সুযোগ বেড়ে যায়।" উত্তরাধিকার-স্কোপ মানে প্রতিটি কুয়েরিতে ভাবতে হবে।

ধাপ ১০

ERD থেকে কোডে — তীরগুলো কোথায় থাকে

`related_name` ছাড়া উল্টো দিকের কুয়েরি লেখা কঠিন

```
class Enrollment(TimeStamped):
    workspace = models.ForeignKey(
        Workspace, on_delete=models.CASCADE,
        related_name="enrollments")
    student = models.ForeignKey(
        StudentProfile, on_delete=models.CASCADE,
        related_name="enrollments")
    class_group = models.ForeignKey(
        ClassGroup, on_delete=models.CASCADE,
        related_name="enrollments")
    is_active = models.BooleanField(default=True)

# ERD-র 1:N তীর মানে কোডে:
#   এক ছাত্র  →  student.enrollments.all()
#   এক শ্রেণি  →  cls.enrollments.filter(is_active=True)
```

তীর = ForeignKey

সন্তান-টেবিলে পিতার id থাকে। ERD-র "N" দিকের টেবিলে কলাম বসে।

উল্টো দিক = related_name

নাহলে `enrollment_set` — পড়তে কষ্ট, খুঁজতেও।

তিনটি FK = একই টেবিলে

Enrollment-এ workspace, student, class — তিনটাই দরকার, তিনটাই আছে।

শিক্ষকের নোট: প্রশ্ন করুন: "workspace কলাম থাকা সত্ত্বেও student আর class_group কেন দরকার?" উত্তর: এক ছাত্র একবার এক শ্রেণিতে — ternary relationship.

সতর্কতা

ERD-তে পাঁচটি সাধারণ ভুল

প্রতিটির প্রতিকার একটি বাক্যে

ভুল	পরিণতি ও প্রতিকার
১. ইতিহাস মুছে ফেলা — ছাত্রের টেবিলে একটাই `class_id`	শ্রেণি বদলালে গত বছরের রেকর্ড হারায় → Enrollment টেবিল
২. কার্ডিনালিটি না লেখা	ছবি পড়া যায় না — "এক শ্রেণিতে কতজন?" প্রশ্নের উত্তর ছবিতেই থাকা দরকার
৩. ঘটনাকে M:N ভাবা	উপস্থিতি/নম্বর/জমা = ঘটনা, নিজের টেবিল ও নিজের ফিল্ড চায়
৪. যৌগিক unique ভুলে যাওয়া	একই পরীক্ষায় দুইবার নম্বর → `uniq_mark_exam_student`
৫. স্কোপ কলাম ছাড়া টেবিল যোগ করা	নতুন টেবিলে `workspace` না দিলে এক প্রতিষ্ঠানের ডেটা আরেকটিতে দেখা যাবে

পঞ্চম ভুলটাই সবচেয়ে বিপজ্জনক — কারণ এটি এরর দেয় না, **চুপচাপ ভুল ডেটা দেখায়**।

শিক্ষকের নোট: শেষ লাইনটা বলুন ধীরে: "নিরাপত্তার ভুল কখনো চিৎকার করে না।" তারপর ছাত্রদের নিজের প্রজেক্টে নতুন টেবিলে workspace আছে কি না দেখতে বলুন।

অনুশীলন

২২ মিনিট — নিজের সিস্টেমের ERD

IMS-এর কাঠামো নকল নয়, পদ্ধতি কপি করুন

১. টেবিলের তালিকা (৫ মিনিট)

নিজের সব মডেল লিখুন — পাশে কোনটি "ঘটনা" (উপস্থিতি, পেমেন্ট, জমা) আর কোনটি "তথ্য" (ছাত্র, শ্রেণি)।

২. সম্পর্ক আঁকুন (৮ মিনিট)

প্রতিটি রেখায় 1:N / M:N / 1:1 লিখুন। যে রেখায় সংখ্যা লিখতে পারেন না — সেখানে বোঝাপড়া এখনো হয়নি।

৩. পিতৃ-সন্তান যাচাই (৫ মিনিট)

প্রতিটি N-দিকের টেবিলে পিতার FK কলাম আছে? উল্টো দিকের `related_name` লেখা?

৪. ইতিহাস পরীক্ষা (৪ মিনিট)

একটি রেকর্ড বদলালে পুরোনো অবস্থা হারায় কি? হারালে মাঝের টেবিল বা `is_active` দরকার।

যাচাই: আপনার ERD দেখে কেউ বলতে পারবে "এই ছাত্র গত বছর কোন শ্রেণিতে ছিল"? না পারলে উত্তরটা কোথায় থাকবে ঠিক করুন।

শিক্ষকের নোট: চতুর্থ ধাপে ছাত্ররা প্রথমবার বুঝবে কেন IMS-এ `is_active` আছে — মুছে ফেলা মানে ইতিহাস হারানো।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- শিক্ষার্থীর শ্রেণি কেন শিক্ষার্থী-টেবিলে কলাম হিসেবে থাকতে পারে না?
- Membership না থাকলে একজন শিক্ষক দুই প্রতিষ্ঠানে কাজ করতে পারতেন না — কেন?
- কোন তথ্য 1:1 টেবিলে রাখা উচিত, আর কোনটা মূল টেবিলেই?
- একই পরীক্ষায় একই ছাত্রের দুইটি নম্বর ঢুকে গেলে সমস্যাটা কোথায় ধরা পড়বে — কোডে, নাকি ডেটাবেসে?

শিক্ষকের নোট: চতুর্থ প্রশ্নের সেরা উত্তর: "ডেটাবেসে — কারণ কোড ভুলে যেতে পারে, constraint ভোলে না।" এটাই ক্লাস ১৫-এর সেতু।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
ঘটনা আর তথ্য আলাদা টেবিলে	Enrollment · AttendanceRecord · Mark · Submission
ভূমিকা User-এ নয়, Membership-এ	এক ব্যবহারকারী বহু প্রতিষ্ঠানে, ভিন্ন ভূমিকায়
1:1 কেবল ঐচ্ছিক বা ভিন্ন দায়িত্বের তথ্যে	Subscription · Policy · Branding · StudentProfile.account
M:N-এ সম্পর্কের নিজের গুণ থাকলে নিজের টেবিল	একটাই Django M:N (LessonLog ⇄ Topic)
PK অদৃশ্য, মানুষের নম্বরটি unique কলাম	id + `(workspace, admission_no)`
নতুন টেবিলে স্কোপ কলাম না থাকলে বিপদ	২৩টি সরাসরি · ৮টি উত্তরাধিকারে

শিক্ষকের নোট: প্রথম ও ষষ্ঠ সারি বোর্ডে আলাদা করে লিখুন — এ দুটোই ছাত্রদের প্রজেক্টে সবচেয়ে বেশি ভুল হয়।

১৫

পরের ক্লাস — "স্কিমা মাটিতে নামে"

partial unique index · workspace কলাম কোথায় নেই · cascade নিয়ম · ১২ মাইগ্রেশনের ক্রম

পর্ব ৩ · ডিজাইন · ক্লাস ১৫ / ৩৭ · শেষ ক্লাস

স্কিমা মাটিতে নামে

সিলেবাস: প্র্যাকটিক্যাল ৪ · টপিক ৪.৩ (ডেটাবেস ডিজাইন)
১২ মাইগ্রেশন · partial unique · cascade · ইনডেক্স মানচিত্র

PostgreSQL · ১২ মাইগ্রেশন · ২৩২ ফিল্ড

শুরু করুন প্রশ্ন দিয়ে

ডেটাবেসে ছবি আর ডেটা — কোথায় থাকে?

হাত তুলুন: ডেটাবেসে? ফোল্ডারে? দুই জায়গায়?

স্কিমা মানে শুধু টেবিল নয় — নিয়ম, ইনডেক্স, ফাইল আর ব্যাকআপের ঠিকানা। ভুল হলে ডেটা হারায়, ধীরে চলে, বা চুপচাপ দ্বিগুণ হয়।

আজকের ক্লাসে ৩১টি টেবিলকে **মাটিতে নামানো** হয়: ১২টি মাইগ্রেশন, কোথায় partial unique লাগে, কোথায় ডিলিট আটকাতে হয়, আর কোন কুয়েরির জন্য কোন ইনডেক্স।

শিক্ষকের নোট: উত্তর সাধারণত "ডেটাবেসে" আসে। বলুন: "লোগো ফাইলটা ডেটাবেসে নেই — ডেটাবেসে আছে তার নাম। ফাইল আর নাম আলাদা জায়গায় থাকলে কী হয়, আজ শেষে দেখবেন।"

ধাপ ১

১২টি মাইগ্রেশন — ক্রমটাই ইতিহাস

৬টি অ্যাপে ১টি করে, ৩টি অ্যাপে ২টি করে

মাইগ্রেশন	কী করে
accounts 0001	User (email-ভিত্তিক লগইন)
workspaces 0001	Workspace · Membership
workspaces 0002	**WorkspaceBranding** — পরে যোগ
academics 0001	বছর · টার্ম · বিভাগ · শ্রেণি · বিষয় · বণ্টন
academics 0002	**CoordinatorScope** — পরে যোগ
students 0001	StudentProfile · Enrollment
attendance 0001	AttendanceRecord · Policy
lessons 0001	৭টি টেবিল — সবচেয়ে বড়
assessments 0001	GradeScale · Band · Exam · Mark
audit 0001	AuditLog + ২টি ইনডেক্স
billing 0001	Plan · Subscription · Invoice · SignupRequest
billing 0002	**ডেটা** মাইগ্রেশন — প্ল্যান বপন + grandfather

মাইগ্রেশন ফাইল মুছবেন না, এডিটও করবেন না। নতুন পরিবর্তন = **নতুন নম্বর**।

একই নম্বর দুই জায়গায় থাকলে `migrate` থেমে যায় — ইতিহাস দ্বিধায় পড়ে।

কোডের ইতিহাস git-এ থাকে; ডেটাবেসের ইতিহাস মাইগ্রেশনে। দুটোই রাখতে হয় — একটার উপর ভরসা করলে অন্যটা হারায়।

শিক্ষকের নোট: বোর্ডে ৬+৬=১২ লিখুন। বলুন: "৩টি অ্যাপে ২টি করে মাইগ্রেশন — কারণ ওই তিনটে জিনিস (Branding, Scope, Plans) শুরুর স্কোপে ছিল না; পরে যোগ হয়েছে।"

ধাপ ২

একটি ডেটা মাইগ্রেশন — deploy যেন অর্ধেক না থাকে

`billing/0002_seed_plans_and_grandfather.py` — স্কিমা নয়, **সারি** তৈরি করে

```
def forwards(apps, schema_editor):
    Plan = apps.get_model("billing", "Plan")
    Subscription = apps.get_model("billing", "Subscription")
    Workspace = apps.get_model("workspaces", "Workspace")

    for order, (code, name, ...) in enumerate(PLANS, 1):
        Plan.objects.update_or_create(code=code, defaults={...})

    for ws in Workspace.objects.filter(kind="INSTITUTION"):
        if not Subscription.objects.filter(workspace=ws).exists():
            Subscription.objects.create(
                workspace=ws, plan=campus, status="ACTIVE",
                current_period_end=today + timedelta(days=365),
                notes="Grandfathered at launch...")

class Migration:
    dependencies = [("billing", "0001_initial"),
                    ("workspaces", "0002_workspacebranding")]
```

কেন মাইগ্রেশনে, স্ক্রিপ্টে নয়

প্রতিটি পরিবেশে একই ক্রমে একবারই চলে — কেউ ভুলে যেতে পারে না।

grandfather মানে কী

পুরোনো প্রতিষ্ঠানকে এক বছরের জন্য সক্রিয়, বিনামূল্যে — ব্যবসার সিদ্ধান্ত কোডে লেখা।

নির্ভরতা অন্য অ্যাপে

`workspaces 0002`-এর পরে — কারণ Branding আগে তৈরি হতে হবে।

নিয়ম: ডেটা মাইগ্রেশন **idempotent** বানান (`update_or_create` , `exists()` চেক) — দুইবার চালালেও ক্ষতি নয়।

শিক্ষকের নোট: প্রশ্ন করুন: "সাবস্ক্রিপশন ছাড়া প্রতিষ্ঠান থাকলে কী হয়?" উত্তর: চিরকাল ফ্রি — অর্থাৎ রাজস্ব হারায়। এজন্যই এটা মাইগ্রেশনে বাধ্যতামূলক করা হয়েছে।

ধাপ ৩

partial unique — "একটাই সক্রিয়" নিয়মটা ডেটাবেসে

শর্তসহ unique index · PostgreSQL-এ সত্যিই partial index

```
constraints = [
  models.UniqueConstraint(
    fields=["workspace"],
    condition=Q(is_active=True),
    name="uniq_active_term_per_workspace"),
]

constraints = [
  models.UniqueConstraint(
    fields=["student", "class_group"],
    condition=models.Q(is_active=True),
    name="uniq_active_enrollment_student_class"),
]

constraints = [
  models.UniqueConstraint(
    fields=["custom_domain"],
    condition=~models.Q(custom_domain=""),
    name="unique_custom_domain"),
]
```

নিয়ম	নাহলে কী হতো
এক workspace-এ একটাই সক্রিয় টার্ম	দুই টার্মে উপস্থিতি — ড্যাশবোর্ড দ্বিগুণ
এক ছাত্র এক শ্রেণিতে একবার সক্রিয়	ভর্তি দুইবার → দুইবার হাজিরা
এক ডোমেইন এক প্রতিষ্ঠানের	দুই স্কুল একই ডোমেইনে → কে জিতবে?

partial বলতে বোঝায়: শুধু শর্ত মানা সারিগুলোর মধ্যে ইউনিক। পুরোনো (নিষ্ক্রিয়) সারি যত চান থাকুক।

SQLite-ও partial index বোঝে — তাই `run.sh`-এর ডেমো DB-তেও নিয়মটা কাজ করে।

শিক্ষকের নোট: বলুন: "কোডে `if active: ...` লিখে আটকানো যায়, কিন্তু দুজন একসাথে সেভ করলে দুইটাই ঢুকে যায়। ডেটাবেস আটকালে সেটা অসম্ভব।" — race condition-এর প্রথম পাঠ।

ধাপ ৪

cascade নিয়ম — ডিলিট করলে কী হয়

৭৫ সম্পর্কে তিনটি পছন্দ: CASCADE ৫১ · SET_NULL ১৭ · PROTECT ৬

CASCADE · ৫১

পিতা গেলে সন্তানও যায়

Workspace মুছলে তার শ্রেণি, ছাত্র, উপস্থিতি, নম্বর — সব যায়

টেন্যান্ট মুছে ফেলা = ইচ্ছাকৃত ধ্বংস

SET_NULL · ১৭

পিতা গেলে সন্তান থাকে, রেফারেন্স ফাঁকা

যে শিক্ষক অ্যাকাউন্ট মুছলেন, তার দেওয়া উপস্থিতি থেকে যায়

recorded_by = NULL

PROTECT · ৬

ডিলিটই হতে দেয় না

টার্ম · প্ল্যান · ইনভয়েসের প্রতিষ্ঠান · প্রতিষ্ঠানের মালিক — আটকে দেয়

নথি কখনো অনাথ হয় না

নিয়ম: **অর্থ** থাকলে CASCADE · **ইতিহাস** মূল্যবান হলে SET_NULL · **নথি** হলে PROTECT। একটি শব্দে: "এই সারিটা না থাকলে অন্যটার মানে থাকে কি?"

শিক্ষকের নোট: উদাহরণ দিন: "সাবস্ক্রিপশনে ব্যবহৃত প্ল্যান মুছলে আগের ইনভয়েসের দাম কী হবে? তাই PROTECT — প্ল্যান নিষ্ক্রিয় করুন, মুছবেন না।"

ধাপ ৫

PROTECT-এর ছয়টি জায়গা — কেন এগুলোই

প্রতিটির পেছনে একটি করে ব্যবসার নিয়ম

সম্পর্ক	যে নিয়মটি রক্ষা করে
ClassGroup.term	টার্ম মুছলে শ্রেণি অনাথ হয়ে যেত — উপস্থিতি কার বিরুদ্ধে?
Exam.term	ফলাফল কোন টার্মের, তা না থাকলে রিপোর্ট কার্ড অর্থহীন
Subscription.plan	চালু চুক্তির প্ল্যান মুছে ফেলা যাবে না — নিষ্ক্রিয় করুন
Invoice.workspace	আর্থিক নথি কখনো প্রতিষ্ঠানহীন হয় না
Invoice.plan	পুরোনো ইনভয়েসে দাম ও প্ল্যান অপরিবর্তিত থাকে
Workspace.owner	মালিক অ্যাকাউন্ট মুছলে প্রতিষ্ঠান ঝুলে থাকত

ভুল সমাধান: PROTECT এড়াতে কোডে `try/except` — নিয়মটা তখন অকার্যকর।

ঠিক সমাধান: `is_active=False` — রেকর্ড থাকে, তালিকা থেকে সরে যায়।

শিক্ষকের নোট: "মুছুন" আর "নিষ্ক্রিয় করুন" — দুটো আলাদা কাজ। বলুন: "শিক্ষক চলে গেলেন, তালিকা থেকে সরান — কিন্তু তাঁর দেওয়া নম্বর থাকবে।"

ধাপ ৬

ইনডেক্স মানচিত্র — ১৩টি নিজের, ২টি কলামে

প্রতিটি ইনডেক্স একটি নির্দিষ্ট কুয়েরির জন্য — অকারণ ইনডেক্স লিখন ধীর করে

ইনডেক্স	কোন পর্দা
`[ws, date]` · attendance	আজকের উপস্থিতি
`[class_group, date]` · attendance	শ্রেণির রেজিস্টার
`[ws, class_group, due_at]`	খোলা হোমওয়ার্ক
`[ws, term, is_published]`	অপ্রকাশিত পরীক্ষা
`[ws, term, is_archived]`	চলতি শ্রেণি তালিকা
`[ws, status]` · students	শিক্ষার্থী তালিকা
`[ws, class_group, is_active]`	সক্রিয় ভর্তি
`[ws, role, is_active]`	সদস্য তালিকা
`[user, is_active]`	আমার প্রতিষ্ঠানসমূহ
`[ws, -created_at]` · `[action]`	অডিট লগ
`[ws, user, is_active]`	কোঅর্ডিনেটর স্কোপ
`[kind, is_active]`	প্রতিষ্ঠান তালিকা

প্রায় সব ইনডেক্স `workspace` দিয়ে শুরু (`ws`) — কারণ প্রতিটি কুয়েরিতেই প্রথম শর্ত প্রতিষ্ঠান।

db_index ২টি: `User.email` (লগইন) · `WorkspaceBranding.custom_domain` (প্রতিটি অনুরোধে হোস্ট দেখে)।

ForeignKey কলামে ইনডেক্স Django নিজেই দেয় — সেটি এই ১৩টির হিসাবে নেই।

শিক্ষকের নোট: বলুন: "ইনডেক্স হলো বইয়ের সূচি। প্রতিটি অধ্যায়ে সূচি রাখলে বই ভারী হয় — তাই যেখানে খুঁজবেন, শুধু সেখানে।"

ধাপ ৭

ইনডেক্স ছাড়া কী হয় — একটি সংখ্যা দিয়ে বোঝা

একই কুয়েরি, দুই স্কিমা: বাড়ির কাজ হিসেবে চালান

```
-- কোন ইনডেক্স ব্যবহার হলো?
EXPLAIN ANALYZE
SELECT * FROM attendance_attendancerecord
WHERE workspace_id = 1 AND date = '2026-11-05';

-- ইনডেক্স থাকলে:
--   Index Scan using attendance_workspace_date_idx
--   Planning: 0.2 ms   Execution: 0.4 ms

-- ইনডেক্স ছাড়া (বড় টেবিলে):
--   Seq Scan on attendance_attendancerecord
--   Rows Removed by Filter: 480000
--   Execution: 180 ms
```

১. EXPLAIN পড়ুন

Seq Scan মানে পুরো টেবিল পড়ছে — বড় টেবিলে সেটাই সমস্যা।

২. কলামের ক্রম মানে

`[workspace, date]` — প্রথমে workspace না দিলে ইনডেক্স কাজে লাগে না।

৩. আগে মাপুন, পরে যোগ

অনুমানে ইনডেক্স দিলে লেখা ধীর হয়, লাভ শূন্য।

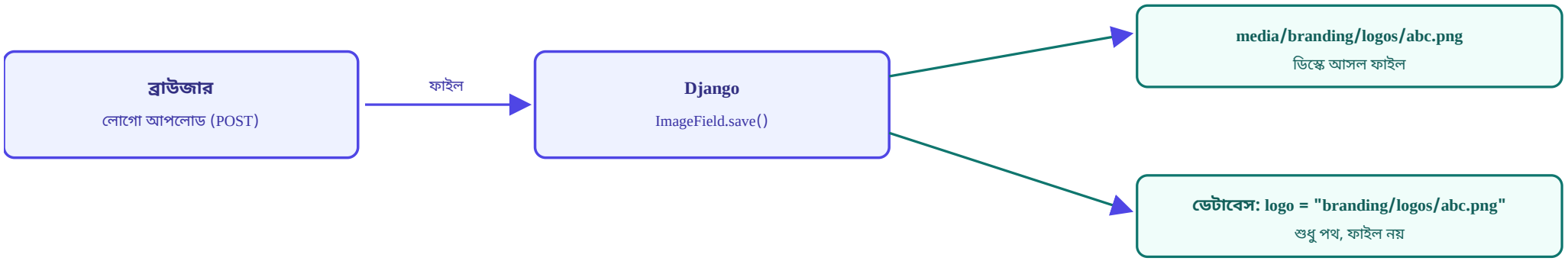
নিয়ম: ইনডেক্স ঠিক করে **পড়া** দ্রুত হয়, কিন্তু প্রতিটি **লেখা** একটু ধীর হয়। তাই প্রতিটি ইনডেক্সের জবাবদিহি থাকা দরকার।

শিক্ষকের নোট: সংখ্যাগুলো উদাহরণ — বলুন স্পষ্ট করে: "এগুলো আপনার ডেটার সংখ্যা নয়, তুলনার জন্য। নিজের টেবিলে EXPLAIN চালিয়ে আসল সংখ্যা দেখুন।"

ধাপ ৮

ফাইল কোথায় থাকে — ডেটাবেসে শুধু নাম

`MEDIA_ROOT` = `BASE_DIR / "media"` · লোগো যায় `branding/logos/`-এ



পরিণতি: ডেটাবেস ব্যাকআপ নিলে ছবি আসে না — media ফোল্ডার আলাদা ব্যাকআপ করতে হয়।

৪টি সেটিংস ফাইল: base (সব) · dev · prod · sqlite (ডেমো, `run.sh`)।

prod-এ ডেটাবেস: PostgreSQL · `CONN_MAX_AGE=60` · media = FileSystemStorage।

শিক্ষকের নোট: প্রশ্ন করুন: "সার্ভার বদলালে ছবি হারাবে কেন?" উত্তর: ডেটাবেস কপি হয়েছে, media ফোল্ডার নয়। এটাই সবচেয়ে সাধারণ ডিপ্লয়-দুর্ঘটনা।

ধাপ ৯

দুই ডেটাবেস, এক স্কিমা

ডেমো SQLite-এ, আসল PostgreSQL-এ — কিন্তু কোড এক, মাইগ্রেশন এক

বিষয়	sqlite (ডেমো)	postgresql (আসল)
কে ব্যবহার করে	`bash run.sh` — নতুন শিক্ষার্থী	প্রতিষ্ঠানের সার্ভার
সেটিংস	`config.settings.sqlite`	`config.settings.prod`
partial unique	কাজ করে (partial index)	কাজ করে
একসাথে লেখা	এক লেখক — সহজ	অনেক লেখক — লক ও transaction দরকার
ব্যাকআপ	একটি ফাইল কপি	`pg_dump` + media ফোল্ডার

শিক্ষার নিয়ম: SQLite দিয়ে শুরু করুন, কিন্তু **নিয়মগুলো PostgreSQL-এর মতোই লিখুন** — constraint, index, transaction। পরে ডেটাবেস বদলালে কোড বদলাতে হবে না।

শিক্ষকের নোট: বলুন: "SQLite খারাপ নয়, ছোট কাজে সেরা। কিন্তু যেখানে একসাথে ৩০ জন শিক্ষক লিখবেন, সেখানে দরকার সার্ভার-ডেটাবেস।"

ধাপ ১০

মাইগ্রেশন-দিনের চেকলিস্ট

স্কিমা বদলানোর আগে-পরে যে পাঁচটি কাজ ভুললে বিপদ

#	কাজ	কেন
১	ব্যাকআপ (`pg_dump` বা ফাইল কপি)	মাইগ্রেশন ফেরানো কষ্টকর, ব্যাকআপ সহজ
২	`makemigrations` আউটপুট পড়ুন	স্বয়ংক্রিয় ফাইলও ভুল হতে পারে — কলাম মুছে ফেলা সহ
৩	নতুন কলাম `null=True` বা default দিন	পুরোনো সারিতে মান ছাড়া কলাম যোগ করা যায় না
৪	ডেটা মাইগ্রেশন আলাদা ফাইলে	স্কিমা আর ডেটা একসাথে গুলিয়ে গেলে ফেরানো কঠিন
৫	মাইগ্রেশনের পরে `103` টেস্ট চালান	স্কিমা বদলালে লুকানো ভাঙন ধরা পড়ে

সবচেয়ে বিপজ্জনক: **কলামের নাম বদলানো** — Django পুরোনো কলাম ফেলে নতুন বানায়, ডেটা রক্ষা পায় না। সমাধান: নতুন কলাম যোগ → ডেটা কপি → পুরোনো কলাম বাদ (তিন ধাপ)।

শিক্ষকের নোট: তৃতীয় সারিটি বোর্ডে লিখুন। বলুন: "প্রোডাকশনে ৫০০০ সারি আছে, নতুন কলাম NOT NULL — migrate মুহূর্তেই থেমে যাবে।"

অনুশীলন

২০ মিনিট — নিজের স্কিমার নিয়ম লিখুন

কোড নয়, সিদ্ধান্ত — তারপর কোড

১. three "একটাই সক্রিয়" নিয়ম খুঁজুন (৫ মিনিট)

আপনার সিস্টেমে কোথায় "একসাথে দুইটি চলতে পারে না"? প্রতিটির জন্য partial unique লিখুন।

২. cascade সিদ্ধান্ত (৫ মিনিট)

প্রতিটি FK-এর পাশে লিখুন: CASCADE / SET_NULL / PROTECT — এবং এক শব্দে কারণ।

৩. ইনডেক্স তালিকা (৬ মিনিট)

আপনার ৫টি সবচেয়ে বড় তালিকা-পর্দা লিখুন; প্রতিটির WHERE-এর কলামগুলো ক্রমে সাজান।

৪. ব্যাকআপ বাক্য (৪ মিনিট)

এক বাক্যে লিখুন: "আমার সিস্টেমের ব্যাকআপ মানে ___।" media ফোল্ডার ভুলে যাবেন না।

যাচাই: আপনার স্কিমায় এমন কোনো নিয়ম আছে কি যা শুধু কোডে লেখা, ডেটাবেসে নয়? থাকলে সেটাই আজকের কাজ।

শিক্ষকের নোট: শেষ প্রশ্নটাই মূল: "কোডে লেখা নিয়ম ভুলে যাওয়া যায়, ডেটাবেসে লেখা নিয়ম ভোলে না।"

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

পর্ব ৩-এর শেষ আলোচনা

- partial unique কেন সাধারণ unique-এর চেয়ে ভালো, যখন ইতিহাস রাখতে হয়?
- শিক্ষক অ্যাকাউন্ট মুছলে তাঁর দেওয়া নম্বর থেকে যাওয়া উচিত — কোন on_delete সেটি করে?
- প্রতিটি ইনডেক্স `workspace` দিয়ে শুরু কেন?
- ডেটা মাইগ্রেশন আর স্কিমা মাইগ্রেশন আলাদা না রাখলে কী কী সমস্যা হয়?

শিক্ষকের নোট: দ্বিতীয় প্রশ্নের উত্তর SET_NULL — এবং এর পরের প্রশ্ন করুন: "তাহলে 'কে দিয়েছিল' জানা যাবে না?" উত্তর: অডিট লগে থাকে। তিনটি স্তর মিলেই সত্য।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
মাইগ্রেশন = ডেটাবেসের git ইতিহাস	১২টি ফাইল · ৩টি অ্যাপে দ্বিতীয় মাইগ্রেশন
নিয়ম ডেটাবেসে রাখুন, কোডে নয়	৪টি partial unique index
ডিলিটের তিন পছন্দ, কারণসহ	CASCADE ৫১ · SET_NULL ১৭ · PROTECT ৬
ইনডেক্স প্রতি কুয়েরির জবাবদিহি	১৩টি Meta index · ২টি db_index · সবই workspace-first
ফাইল ডিস্কে, পথ ডেটাবেসে	`media/` আলাদা ব্যাকআপ লাগে
এক স্কিমা, দুই ডেটাবেস	sqlite (ডেমো) · postgresql (আসল)

শিক্ষকের নোট: পর্ব ৩ শেষ। বলুন: "ডিজাইন শেষ মানে কোড লেখা শুরু নয় — ডিজাইন শেষ মানে এখন কোড লেখার সিদ্ধান্তগুলো আর বদলাবে না।"

৪

পর্ব ৪ — "কোড লেখা শুরু"

ক্লাস ১৬: মডেল লেখা — ৩১টি টেবিল Django-তে · ফিল্ডের ধরন · migration-এর অভ্যাস